

Kafka, in Python

A visual, example-driven tour of distributed streaming

Jonghwan Lee

2026

Contents

Preface	8
Who this book is for	8
How to read this book	8
Conventions	9
A note on reproducibility	9
Acknowledgements	10
 What Kafka Is	 10
The one-picture explanation	10
Why another messaging thing?	11
Records, topics, partitions	12
A tiny end-to-end trip	13
What Kafka is not	13
The three things to take from this chapter	14
Exercises	14
 Topics, Partitions, Offsets	 15
A partition is a file, more or less	15
How a record finds its partition	16
Offsets, high-water marks, and log-end	18
Listing what's really there	19
Choosing a partition count	19
Exercises	20

A Local Kafka in Three Minutes **20**

The single-broker docker setup	20
Installing a Python client	22
Your first producer	22
Your first consumer	23
Common first-run surprises	25
Tearing it down	26
Exercises	26

Producing, Properly **26**

The producer loop	27
Acknowledgements: <code>`acks`</code>	28
Idempotent producer	29
Batching and compression	29
Keys, partitioning, and hot spots	30
A production-shaped producer	31
Exercises	32

Consuming, Properly **32**

Consumer groups	32
The three positions a consumer tracks	34
Committing explicitly	35
Rebalances	36
<code>`seek()`</code> , and when to use it	37
A resilient consumer skeleton	37
Exercises	38

Delivery Semantics **39**

The three semantics, concretely	39
Where duplicates actually come from	41
Idempotent producer (already on)	41
Transactions: atomic write-and-commit	41
Exactly-once in practice	43
A worked comparison	43
Exercises	44

Schemas and the Registry **45**

The three serialisation options	45
An Avro record, from Python	46
Schema evolution	47
A compatible evolution in practice	49
Schema-less pragmatism	50
Exercises	50

Stream Processing **51**

What stream processing actually adds	51
Approach 1: Faust-streaming	52
Approach 2: Bytewax	53
Approach 3: do it yourself	54
When <i>*not*</i> to reach for a streaming framework	55
Exercises	56

Kafka Connect **56**

The shape of Connect	56
--------------------------------	----

Running Connect locally	57
Source connector example: a file	58
Sink connector example: S3	59
CDC: the killer use case	60
When <i>*not*</i> to use Connect	61
Monitoring Connect	62
Exercises	62

Replication and Durability 62

Replicas, leaders, followers	63
The ISR (in-sync replica set)	64
The high-water mark, revisited	64
Unclean leader election	65
Rack awareness	65
Reassignment: moving partitions	66
Durability at the broker	66
A durability checklist	67
Exercises	67

Performance and Tuning 68

Four numbers that matter	68
The first rule: measure	69
The throughput–latency knobs	69
Partition count	70
Broker-side	71
The memory story	71
What a running broker looks like	72

Exercises	72
Operating a Cluster	72
Cluster shape	73
Deploying: the three realistic paths	74
Upgrades	74
Security, briefly	75
Monitoring	75
Quotas	76
Disaster recovery	76
Day-to-day runbooks	77
Exercises	77
Patterns That Keep Working	78
Event sourcing	78
CQRS	79
Outbox	80
Compacted topic as a key-value store	80
Dead-letter queues	81
Saga pattern	82
Exactly-once stream processing	82
Tiered storage	83
The anti-patterns	83
Exercises	83
A Case Study: The Feed Pipeline	84
The shape	84

Step 1: event envelope	85
Step 2: outbox + CDC	86
Step 3: user state, compacted	87
Step 4: the enricher	87
Step 5 & 6: the sinks	89
Step 7: monitoring	89
Failure walk-through	90
Numbers	90
What this skips	91
Exercises	91

Appendix A: The CLI Cheatsheet **92**

Topics	92
Consumer groups	92
Producing and consuming from the shell	93
Cluster metadata	94
Partition reassignment	94
ACLs (authorisation)	95
Configurations	95
GetOffsetShell	96
From Python	96

Appendix B: Glossary **97**

CHAPTER

Preface

Kafka has a reputation for being big, operational, and full of jargon. Brokers, controllers, ISRs, KRaft, replication factors, consumer group coordinators — you can drown in a week of vocabulary before you have produced a single message. That is a shame, because the core of Kafka is one of the cleanest ideas in distributed systems: an append-only log, sharded and replicated, that several readers can scan at their own pace.

This book is a tour of that idea and its consequences, with Python at your fingertips the whole way. We start with the shape of a single partition, produce one message, consume it, and only then do we reach for a second broker. By the time we get to exactly-once semantics, you will know what it is trying to protect you from — because you will already have seen the failure modes it is designed around.

Who this book is for

You should be comfortable with Python and with writing a small service that talks to the network. You do not need prior experience with Kafka or with any message broker. If you have used Redis pub/sub, RabbitMQ, or SQS, you will notice where the analogies hold and where they don't; where they don't is usually where Kafka earns its keep.

You do not need to know Java. Every code sample is Python. We will touch on the CLI tools that ship with Kafka (which are Java) only when they are the most direct way to show something.

How to read this book

Every chapter has the same four parts:

1. **Idea** — what the chapter is about, stated in plain prose.
2. **Worked examples** — short Python programs that you can run.
3. **Figures** — one or two diagrams per chapter, generated from code in `scripts/figures/`.

4. **Exercises** — small, specific, solvable in under half an hour.

Chapters 1 through 6 build the mental model. Chapters 7 through 9 add the ecosystem pieces you usually need in production: schemas, stream processing, and connectors. Chapters 10 through 12 are about running Kafka — replication, performance, and the day-to-day of operating a cluster. The last two chapters pull everything together: design patterns that keep showing up on real systems, and an end-to-end case study.

Conventions

Code that you type appears in a monospaced box:

```
from confluent_kafka import Producer

p = Producer({"bootstrap.servers": "localhost:9092"})
p.produce("hello", key="greeting", value="world")
p.flush()
```

Output, when shown, follows immediately below:

```
delivered to hello[0] @ offset 0
```

Three kinds of inline callouts appear throughout the book:

Note

A clarification, a cross-reference, or a useful digression.

Tip

A concrete suggestion — something I've learned to do (or avoid) on real systems.

Warning

A place where the obvious reading is wrong, and the bug is easy to ship to production.

A note on reproducibility

Every figure in this book is produced by a Python script that lives in `scripts/figures/`. Every random seed is fixed, so the numbers and shapes you see on the page are the numbers and shapes you'll see on your laptop. Code examples assume a local Kafka broker on `localhost:9092`; Chapter 3 walks you through getting one running in about three minutes.

Kafka moves quickly: configs get added, defaults get tuned, and new features land in every release. I note the Kafka version I used (3.7) wherever behaviour depends on it. Most of what this book teaches will outlive any specific release.

Acknowledgements

To the colleagues and readers who argued with me about partitioning keys, consumer group rebalancing, and what "exactly once" actually means: thank you. The bugs that remain are mine.

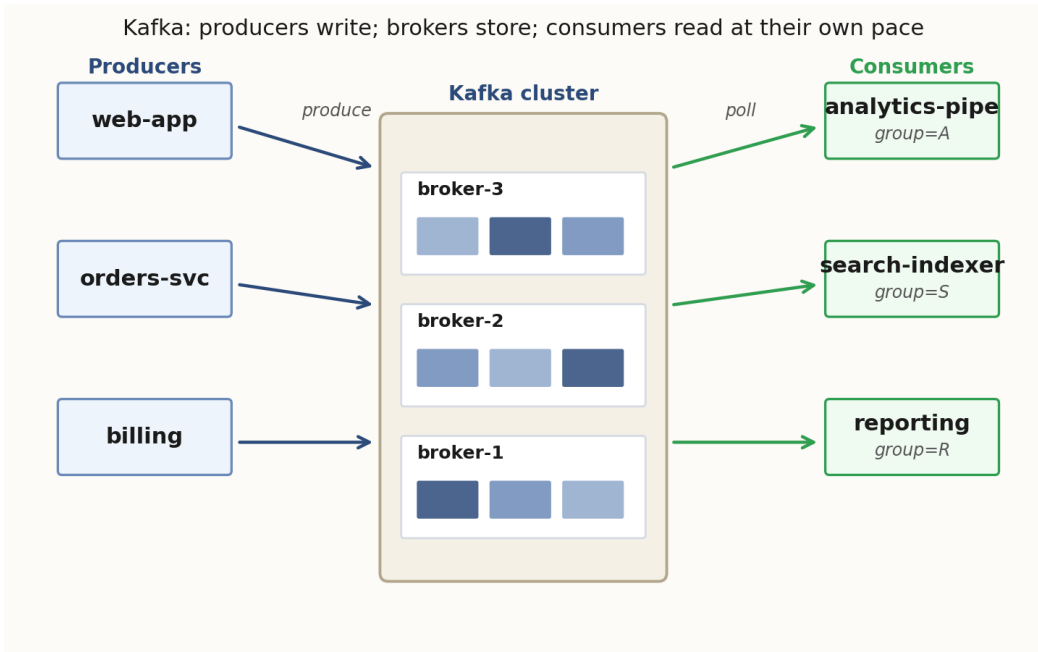
CHAPTER

What Kafka Is

Strip away the documentation, the CLIs, and the brand, and Kafka is a distributed, append-only log. You write records to the end; readers scan from wherever they left off. That is the whole shape of it. Every other feature — partitioning, replication, consumer groups, stream processing — is a consequence of that shape scaled to many machines and many clients.

This chapter makes that sentence concrete. We'll look at what a Kafka cluster looks like from outside, what the core vocabulary is, and what kinds of problems Kafka is actually good at versus the problems it is sometimes wrongly reached for.

The one-picture explanation



Kafka at a glance: producers write records into partitioned, replicated logs; consumers read them at their own pace. Brokers store the logs; Kafka itself is stateless about what consumers have seen.

Three kinds of parties talk to a Kafka cluster:

- **Producers** write records to *topics*.
- **Consumers** read records from topics, tracking their own position.
- **Brokers** are the servers that hold the topic data on disk and replicate it among themselves.

A topic is split into one or more *partitions*; each partition is an ordered, append-only sequence of records. Brokers hold partitions; a producer writes to a partition; a consumer reads from a partition. The ordering guarantee that Kafka provides is per-partition, not per-topic. This is the single most useful sentence you can remember.

Why another messaging thing?

Kafka exists in a space crowded with message brokers and queues: RabbitMQ, ActiveMQ, SQS, Redis streams, NATS, Pulsar, and many more. Each of these has its sweet spot. Kafka's is specifically:

1. **Throughput orders of magnitude higher than traditional brokers**, because it treats the log as the source of truth and never rewrites history to acknowledge consumption.
2. **Replayable history**. Consumers can scroll back to any offset a week ago. Other brokers treat a message as gone once it is acknowledged.
3. **Multiple independent consumers of the same data**. Because the broker doesn't track consumption, adding a second, third, or tenth reader is free.
4. **Durable buffering between systems**. Kafka is the shock absorber between a fast producer and a slow consumer, for hours or days, not seconds.

If you need request/response with strict routing rules, RabbitMQ is usually a better fit. If you need a transactional queue inside one service, your database already has one. Kafka shines when you have many services that want to see the same stream of facts and act on them at different paces.

Records, topics, partitions

A Kafka *record* is a key, a value, a timestamp, and optional headers. Key and value are just byte arrays to Kafka — the broker doesn't care what they mean. Consumers and producers agree on a format (JSON, Avro, Protobuf) between themselves.

```
# A record, roughly
record = {
    "key": b"order-42",
    "value": b'{"item": "coffee", "qty": 1}',
    "timestamp": 1_713_700_000_000,
    "headers": [("trace-id", b"abc-123")],
}
```

Records live in partitions. A partition is a file (well, a sequence of segment files) on a broker disk. Records are identified by an *offset* — a monotonic 64-bit integer that is unique within a partition. Offset 0 is the first record; offset 1 is the second; and so on, forever, even across broker restarts.

Note

The broker stores offsets; consumers store their *committed* offsets (their position) separately. The broker does not know or care which records any particular consumer has read. This single design decision is why Kafka scales to thousands of readers.

A tiny end-to-end trip

Chapter 3 will show you how to install Kafka locally. For now, take it on faith that a broker is running on `localhost:9092` and trace the life of a single record. A producer sends:

```
from confluent_kafka import Producer

p = Producer({"bootstrap.servers": "localhost:9092"})
p.produce("orders", key="42", value='{"item": "coffee"}')
p.flush()
```

The broker writes the record to the end of a partition's log on disk, assigns it offset N , and acknowledges. Later, somewhere else, a consumer asks for "whatever's new on `orders` after offset M ":

```
from confluent_kafka import Consumer

c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id":         "orders-ui",
    "auto.offset.reset": "earliest",
})
c.subscribe(["orders"])
msg = c.poll(1.0)
print(msg.value())
```

Between those two programs, anything can be true. The producer could have crashed after `produce` but before `flush`. The consumer could be running a week later. A second consumer could be tailing the same partition to feed an analytics warehouse. Three brokers could share the partition so that any one of them can die without data loss. Kafka's job is to make the record survive all of that and come out in the right order on the other side.

What Kafka is not

It is worth naming the common misreadings, because each one trips people up on their first real project.

- **Not a database.** You can query the log sequentially, from an offset forward. You cannot say "give me the record with key `order-42` " and have Kafka find it. Stream processors built *on top of* Kafka (Kafka Streams, ksqlDB) build indexed state, but the broker itself doesn't.
- **Not a task queue in the RabbitMQ sense.** Kafka doesn't have per-message acknowledgements, requeues, or priorities. Once a record is in a partition, the only way to skip it is to advance the consumer's offset past it.
- **Not low-latency.** Kafka is built for high-throughput and durable. End-to-end latency is typically tens of milliseconds, sometimes single-digit with care — fine for most streaming, not fine for HFT.
- **Not free.** An idle three-broker cluster still uses real memory and disk. Below a certain load, a shared database queue is simpler, cheaper, and easier to operate.

The three things to take from this chapter

Tip

1. **Kafka is a log, not a queue.** Records stay on disk after they are read.
2. **Ordering is per-partition, never per-topic.**
3. **Consumers track their own position.** The broker only stores the log.

The rest of this book is, in one sense, a series of concrete consequences of those three facts.

Exercises

1. For each of the following systems, argue in two sentences whether Kafka is a natural fit or a forced one: a PostgreSQL replication pipeline; an internal RPC layer; a mobile-app analytics firehose; a work queue for resizing uploaded images.
2. A colleague claims that "Kafka guarantees ordering." Write the most precise version of that statement you can, in one sentence, using the word *partition*.
3. A topic has 3 partitions and 1 million records per day. A second team wants to add a new consumer of the same topic without slowing down the existing one. How much extra *disk* does that cost?

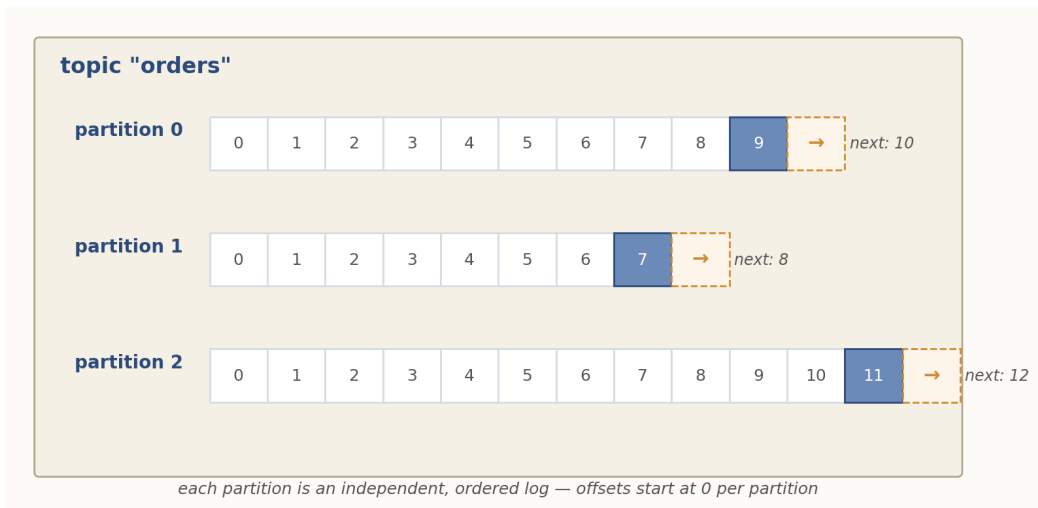
Topics, Partitions, Offsets

The previous chapter said Kafka is a distributed log. This chapter looks at exactly what that log looks like on disk and in the wire protocol, because almost every puzzling piece of Kafka behaviour dissolves once you can picture it.

We will walk through the anatomy of a partition, how a message ends up in one partition and not another, what offsets really are, and how Kafka keeps the log from growing forever. By the end you should be able to answer: "if I write a record right now, where does it go and what gets to read it?"

A partition is a file, more or less

Physically, a partition on a broker is a directory containing several *segment* files. Each segment file holds a contiguous range of offsets. When a segment reaches its configured size (1 GB by default) the broker closes it and opens a new one. Old segments are eventually deleted by the retention policy.



A topic split into 3 partitions. Each partition is an independent, ordered log on disk. Offsets restart at zero per partition, not per topic.

A few things follow from this picture:

- **Append-only.** Records go on the end; existing records never move. An offset, once assigned, never changes meaning.
- **Sequential I/O.** Writes are append-to-file; reads from a cold segment are sequential scans. This is why Kafka is fast on cheap disks: it is doing the thing spinning rust is good at.
- **Deletion is by whole segment.** Kafka doesn't delete individual records; it drops entire segments once they fall outside the retention window.

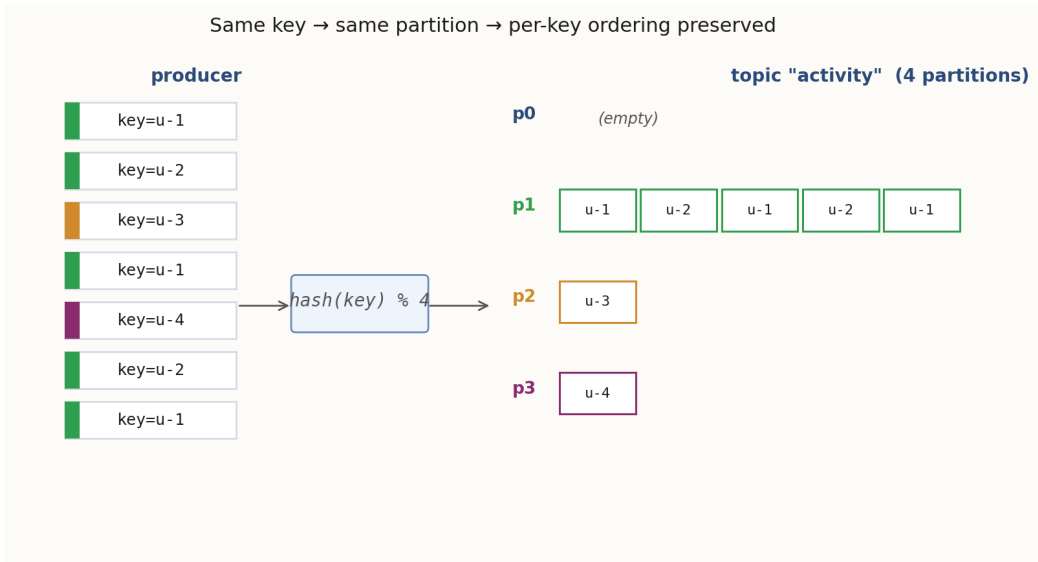
Note

Kafka supports two retention modes: *delete* (drop old segments after a time or size limit) and *compact* (keep only the latest record per key). Delete is the default; compact is what you pick when the log is a changelog of state — more in chapter 13.

How a record finds its partition

When a producer calls `produce("orders", key=k, value=v)`, Kafka has to pick a partition. It follows a simple rule:

1. If the producer explicitly names a partition, use that.
2. Else, if the record has a non-null key, hash the key and take `hash(key) % num_partitions`.
3. Else, round-robin across partitions (with some batching heuristic in modern clients).



Keyed partitioning: the same key always lands in the same partition, preserving per-key ordering. Keyless records fan out by round-robin.

The second rule — hash by key — is the one that matters. It means records sharing a key are guaranteed to share a partition, and therefore to be read in order. If `user_id` is the key, every event for a user is ordered; across users, no ordering is guaranteed. Pick keys with this in mind.

```
from confluent_kafka import Producer

p = Producer({"bootstrap.servers": "localhost:9092"})

# Same user → same partition → per-user ordering preserved.
for event in user_events:
    p.produce("activity",
              key=str(event["user_id"]),
              value=json.dumps(event))

p.flush()
```

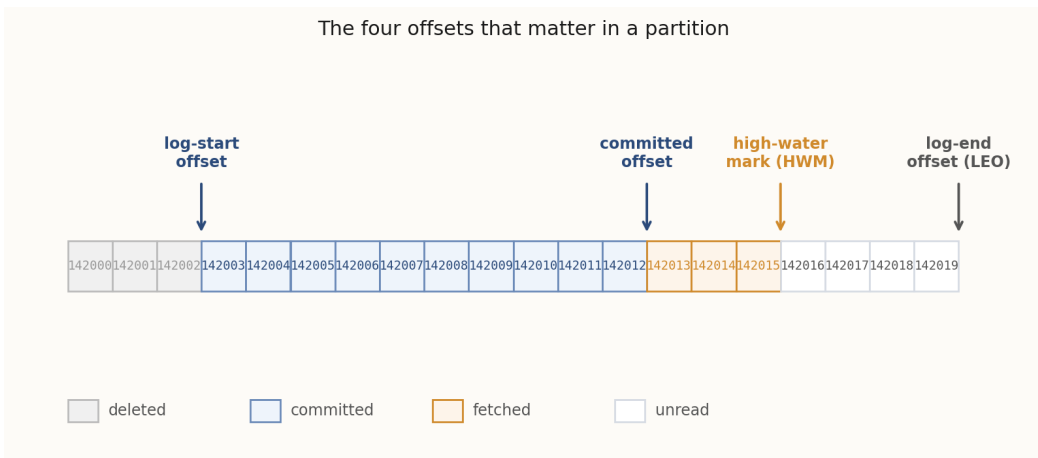
Warning

Adding partitions later **changes** which partition a given key hashes to. Per-key ordering silently breaks at the moment you repartition. Plan partition count up front and leave headroom — see chapter 11.

Offsets, high-water marks, and log-end

Every record in a partition has a 64-bit *offset*. Three offsets per partition matter to clients:

- **Log-start offset.** The smallest offset still on disk. Older records have been deleted by retention.
- **Log-end offset (LEO).** One past the last written offset — the offset the next record will get.
- **High-water mark (HWM).** One past the last record *replicated* to all in-sync replicas. Consumers are only allowed to read up to the HWM. More on this in chapter 10.



Life of an offset inside a partition: records stream in at the log-end offset; consumers advance committed offsets as they process; retention eventually deletes from the left edge.

A consumer tracks two things: its *current position* (what it will read next) and its *committed offset* (what it has promised to remember if it restarts). These are separate, and the distinction is the root of most consumer bugs — we dig in in chapter 5.

Listing what's really there

The Kafka CLI is handy for peeking under the hood. From a broker container:

```
kafka-topics.sh --bootstrap-server localhost:9092 \  
  --create --topic orders --partitions 3 --replication-factor 1  
  
kafka-topics.sh --bootstrap-server localhost:9092 \  
  --describe --topic orders
```

Output:

```
Topic: orders PartitionCount: 3 ReplicationFactor: 1  
  Topic: orders Partition: 0 Leader: 1 Replicas: 1 Isr: 1  
  Topic: orders Partition: 1 Leader: 1 Replicas: 1 Isr: 1  
  Topic: orders Partition: 2 Leader: 1 Replicas: 1 Isr: 1
```

Three partitions, all on broker 1 (we only have one broker). From Python, the same introspection is available through the admin client:

```
from confluent_kafka.admin import AdminClient  
  
admin = AdminClient({"bootstrap.servers": "localhost:9092"})  
md = admin.list_topics(timeout=5)  
  
for name, t in md.topics.items():  
    print(name, "-- partitions:", len(t.partitions))
```

Choosing a partition count

This is the one decision about a topic that is hard to reverse in production, so spend an extra minute on it.

Too few partitions → you cap consumer parallelism. If the topic has 2 partitions, at most 2 members of a consumer group do useful work; the rest sit idle. You also cap producer throughput per key.

Too many partitions → each partition is a file, a network connection target, and a per-partition overhead on the broker. At high partition counts (>4k per broker) you start to see controller slowdowns and longer failover times.

Tip

A good rule of thumb for a new topic: pick `partitions = max(expected consumer parallelism, expected MB/s ÷ per-partition throughput)`. For most services that is somewhere between 6 and 24. Revisit when traffic actually reaches 10× your initial estimate.

Exercises

1. A topic has 6 partitions. You send 1,000 records keyed on `user_id`. How many partitions end up with data? How is the load distributed?
2. A colleague says: "we have 128 partitions on our main topic, so we get ordering by user *and* full throughput." What is wrong, or right, about that statement?
3. The CLI reports `Log start offset = 142_000, High-water mark = 203_500` for a partition. How many records are currently readable from this partition?
4. You add three partitions to a live topic. Events for `user_id = 42` previously went to partition 1. Where do they go now? What does this imply for any consumer that was relying on per-user ordering?

CHAPTER

A Local Kafka in Three Minutes

The rest of this book has running code in it. You need a broker somewhere that speaks the Kafka protocol on `localhost:9092`, plus a Python environment with a client library installed. This chapter walks through both, using only tools that run on your laptop and leave no residue when you're done.

The single-broker docker setup

The simplest reproducible Kafka is a single broker running in a container. Kafka 3.x ships in *KRaft mode* — no ZooKeeper dependency — which makes the docker-compose file pleasantly short.

```
# docker-compose.yml
services:
  kafka:
    image: apache/kafka:3.7.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: \
        PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      CLUSTER_ID: 5L6g3nShT-eMctK--X86sw
```

Bring it up:

```
docker compose up -d
```

Wait a few seconds and confirm the broker is listening:

```
docker compose logs kafka | grep "Kafka Server started"
```

You now have a single-node Kafka cluster on `localhost:9092`. It is enough for every example in this book up to chapter 10, where we switch to a three-broker setup to see replication.

Tip

Single-node Kafka is perfect for development but has no redundancy. `replication.factor=1` means if the broker dies, your topic data is gone. That's fine for experimentation, dangerous in production.

Installing a Python client

There are two actively maintained Kafka clients in Python:

- **confluent-kafka-python** — thin Python binding over librdkafka. Fast, mature, mirror of the Java client. Our default.
- **kafka-python** — pure-Python reimplementation. No C dependency, but slower and historically lags on features.

We use `confluent-kafka` throughout:

```
pip install confluent-kafka
```

The package ships `Producer`, `Consumer`, `AdminClient`, and helpers for Avro / Protobuf. Schema-registry integration lives in `confluent-kafka[avro]` and friends — more in chapter 7.

Your first producer

Create a topic and write a single record:

```
# scripts/hello_producer.py
from confluent_kafka import Producer
from confluent_kafka.admin import AdminClient, NewTopic

BOOTSTRAP = "localhost:9092"

admin = AdminClient({"bootstrap.servers": BOOTSTRAP})
admin.create_topics([NewTopic("hello", num_partitions=1,
                             replication_factor=1)])

p = Producer({"bootstrap.servers": BOOTSTRAP})

def on_delivery(err, msg):
    if err:
        print("delivery failed:", err)
    else:
        print(f"delivered to {msg.topic()} [{msg.partition()}]"
              f" @ offset {msg.offset()}")

p.produce("hello", key="greeting", value="world",
         on_delivery=on_delivery)
p.flush()
```

Run it:

```
$ python scripts/hello_producer.py
delivered to hello[0] @ offset 0
```

The `on_delivery` callback fires once the broker acknowledges the record. `flush()` blocks until all outstanding records either succeed or fail. Never exit a producer process without calling `flush()` — in the async model, `produce` only queues the record, it doesn't send it yet.

Your first consumer

```
# scripts/hello_consumer.py
from confluent_kafka import Consumer

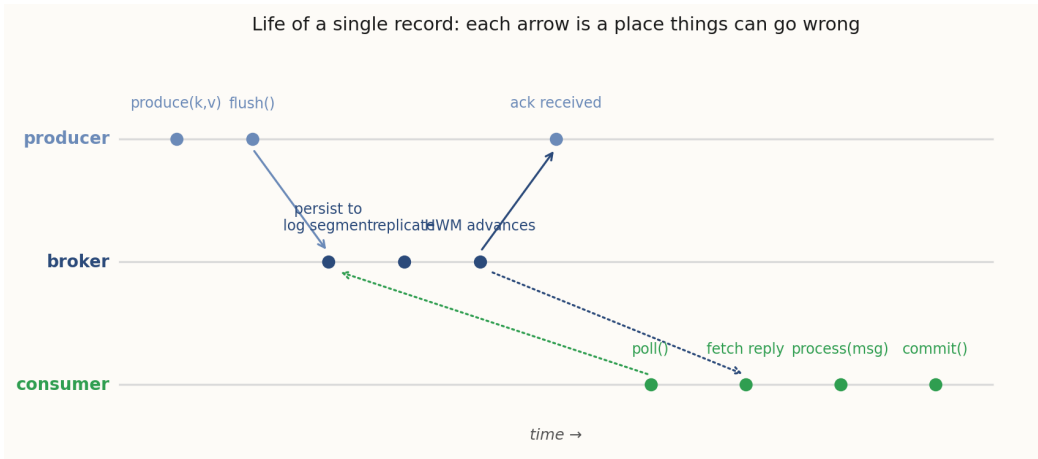
c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "hello-reader",
    "auto.offset.reset": "earliest",
})
c.subscribe(["hello"])

try:
    while True:
        msg = c.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            print("consumer error:", msg.error())
            continue
        print(f"{msg.offset()}: {msg.key()} → {msg.value()}")
finally:
    c.close()
```

Run it in a second terminal:

```
$ python scripts/hello_consumer.py
0: b'greeting' → b'world'
```

Run the producer again and watch the consumer print the new record within a second. This closed loop — producer → broker → consumer — is the picture you should have in your head for the rest of the book.



Common first-run surprises

Three things surprise almost everyone on their first real program:

The consumer reads "nothing" until you produce. An empty topic with a consumer that set `auto.offset.reset=earliest` will happily block on `poll`, because there is no record yet. This is correct behaviour, just disorienting.

Re-running the consumer doesn't re-read old records. Once the consumer group has committed an offset, restarting the process picks up after the last committed position. To start over, either pick a new `group.id` or reset the group:

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group hello-reader --topic hello \
  --reset-offsets --to-earliest --execute
```

Produce is asynchronous. `produce()` does not send; it queues. You *must* `poll()` the producer periodically (or `flush()` at the end) for delivery reports and retries to run. Forgetting `flush()` is the classic Python-Kafka bug.

Warning

Tutorials often leave out `p.flush()` because their code hangs around long enough that the library's background thread flushes on its own. In a short-lived script, that is not guaranteed: your process will exit with unsent records in the buffer.

Tearing it down

When you're done:

```
docker compose down -v
```

The `-v` also removes the volume, so the next `up` starts from a blank slate. Good for learning; bad habit in production, of course.

Exercises

1. Produce three records with keys `"a"`, `"b"`, `"a"` into the `hello` topic. Do they land in the same partition? (You'll need to recreate the topic with `--partitions 3`.)
2. Write a consumer that stops after reading exactly 5 records. (Hint: `poll()` returns `None` when no record is available; exit once your counter reaches 5.)
3. Start the producer without calling `flush()`. Does the consumer see the records? Now add `flush()`. What changes?

CHAPTER

Producing, Properly

A producer in Kafka does more than push bytes. It picks partitions, batches records to hide network latency, negotiates durability with the broker, handles retries on transient failures, and decides whether your program sees an acknowledgement before the next line runs. Each of those is a knob, and most production incidents that touch Kafka trace back to a default that wasn't right for the workload.

This chapter walks through the knobs that matter, shows the default behaviour explicitly, and builds up to a producer that is safe to run in an order-processing service.

The producer loop

The `confluent-kafka` client is asynchronous by design. You call `produce()` to hand a record to the library; the library accumulates records into batches; a background thread ships those batches to brokers and runs your delivery callbacks when they ack.

```
from confluent_kafka import Producer

p = Producer({
    "bootstrap.servers": "localhost:9092",
    "client.id":         "orders-svc",
})

def delivered(err, msg):
    if err:
        # Log and handle; don't raise — this runs on the library thread.
        print("FAIL:", err, msg.key())
    else:
        print(f"OK {msg.topic()} [{msg.partition()}]@{msg.offset()}")

for event in stream_of_events():
    # Block briefly if the local queue is full; never drop silently.
    while True:
        try:
            p.produce("orders",
                      key=str(event["id"]),
                      value=json.dumps(event),
                      on_delivery=delivered)

            break
        except BufferError:
            p.poll(0.1)
    p.poll(0)  # serve delivery callbacks for finished batches

p.flush()
```

Four things in that loop are worth naming:

- `produce()` **can raise** `BufferError` when the internal queue is full. The right reaction is to poll for completions to drain the queue — not to drop the record.

- `poll(0)` runs any delivery callbacks for batches that finished since the last call. Without it, your callbacks never fire.
- `flush()` **at the end** forces all queued records to send before the process exits. Forgetting it is the most common Kafka-Python bug.
- **Callbacks run on the library thread.** Don't raise in them, and don't touch non-threadsafe state.

Acknowledgements: `acks`

`acks` controls when the broker tells the producer "got it":

`acks`

Meaning

Durability

`0`

Don't wait for an ack at all

Weakest

`1`

Wait for the partition leader to write

Moderate

`all`

Wait for all in-sync replicas to write

Strongest

`acks=all` combined with a `replication.factor` of 3 and a `min.insync.replicas` of 2 is the standard durable setup: a record is only acked once it is on two of three broker disks. `acks=1` is a common mistake — it feels durable, but if the leader dies before a follower fetches the record, the record is lost.

```
p = Producer({
    "bootstrap.servers": "localhost:9092",
    "acks": "all",
    "enable.idempotence": True,          # see below
    "compression.type": "zstd",
    "linger.ms": 20,
})
```

Warning

`acks=0` is almost never the right choice. The throughput gain over `acks=1` is small (often single-digit percent) and you lose the ability to know whether anything worked. If you genuinely do not care about the record, don't send it.

Idempotent producer

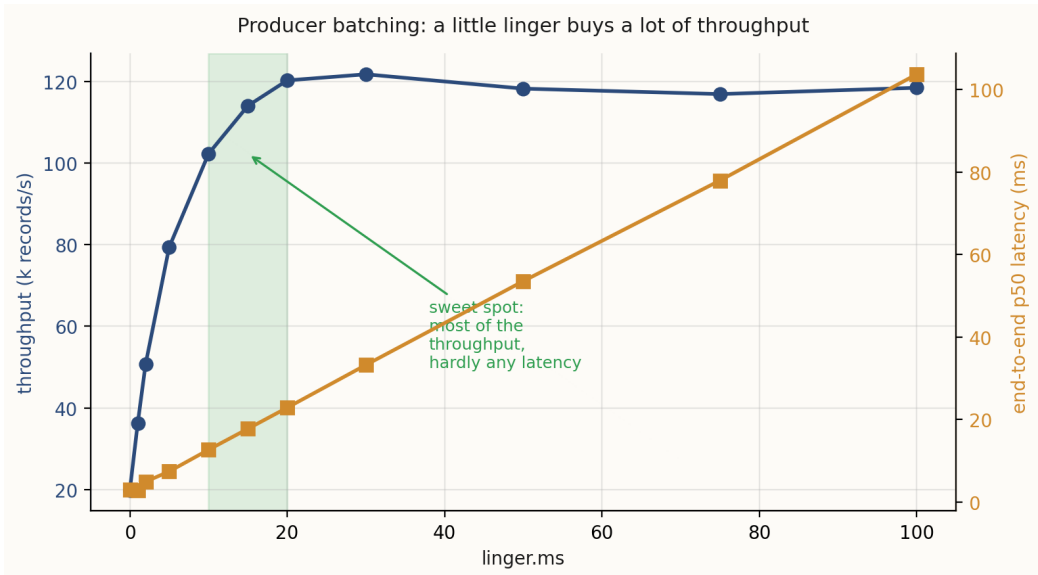
Set `enable.idempotence=True` and the producer attaches a producer ID and sequence number to each batch. The broker uses these to detect and drop duplicate batches caused by retries. The cost is tiny; the benefit is that retries no longer cause duplicates in the log. Turn this on for any non-trivial producer.

With `idempotence` on, Kafka forces `acks=all` and a bounded `max.in.flight.requests.per.connection` (5 in modern versions) so ordering is preserved even under retry.

Batching and compression

Kafka producers batch. A record you `produce()` does not hit the wire immediately; it joins a per-partition batch that the library flushes when any of the following happens:

- `batch.size` bytes have accumulated for that partition, or
- `linger.ms` milliseconds have elapsed since the batch opened, or
- `flush()` is called.



Throughput as a function of `linger.ms` and `batch.size`. A little linger costs a little latency and buys a lot of throughput, up to a point; past roughly 20ms you mostly shave the per-record CPU cost and no more.

The default `linger.ms=0` means "send as soon as you can," which is the worst setting for most producers: batches are tiny, overhead dominates, throughput tanks. Set `linger.ms=10` or `20` for anything doing real traffic — the added latency is imperceptible, and the compression ratio (next paragraph) improves dramatically.

Compression works on whole batches, not individual records. A batch of 100 small JSON records usually compresses 4–8× with `zstd`. That is a direct reduction in bytes on the wire and on disk. `zstd` and `lz4` are the two sensible choices: `zstd` for smaller files, `lz4` for lower CPU.

Keys, partitioning, and hot spots

A key does two things:

1. Routes the record to a partition (`hash(key) % partitions`).
2. Makes log compaction meaningful (chapter 13).

Pick a key whose cardinality is much higher than your partition count and whose distribution is reasonably uniform. Common pitfalls:

- **Hot keys.** If 40% of your traffic is for `user_id = 0` (a system account), one partition carries 40% of the load. Consider a salt or a two-level strategy.
- **Low-cardinality keys.** A key with 4 values across 32 partitions uses at most 4 partitions. Everyone else sits idle.
- **No key at all.** Fine when you don't need per-entity ordering. The producer round-robins in batches (an optimisation called *sticky partitioning*) — records still batch well, they just don't preserve per-entity order because there are no entities.

A production-shaped producer

Wrapping it up — a configuration that is a reasonable starting point for a service sending business events:

```
p = Producer({
    "bootstrap.servers":    "kafka-1:9092,kafka-2:9092,kafka-3:9092",
    "client.id":            "orders-svc",

    # Durability
    "acks":                 "all",
    "enable.idempotence":   True,
    "retries":              10_000_000,
    "delivery.timeout.ms":  120_000,

    # Throughput
    "compression.type":     "zstd",
    "linger.ms":            20,
    "batch.size":           64 * 1024,

    # Fail fast on misconfiguration
    "socket.timeout.ms":    30_000,
})
```

Every field has a reason:

- `acks=all` + idempotence + `replication.factor>=3` → no data loss on single-broker failure.
- `retries` effectively unbounded; `delivery.timeout.ms` is the real deadline — the library will retry transient errors up to 2 minutes before giving up.

- `zstd` compression + 20 ms linger + 64 KB batches is the sweet spot for most services.

Exercises

1. Produce 10,000 records with `linger.ms=0` and 10,000 with `linger.ms=20` . Measure wall-clock time in both cases. Explain the difference.
2. Write a producer that retries on `BufferError` up to 10 times with exponential backoff, then drops the record to a dead-letter file.
3. Set `acks=1` and kill the broker right after a successful produce (hard — do it in a test cluster). Is the record in the topic after a new leader is elected? Why or why not?
4. The delivery callback prints before the next line of `main()` runs. True or false, and why?

CHAPTER

Consuming, Properly

Where producers are about durability, consumers are about *position*. Reading a record and acting on it is easy; the hard part is agreeing with the broker that you have done so, so that a restart doesn't re-read or skip records.

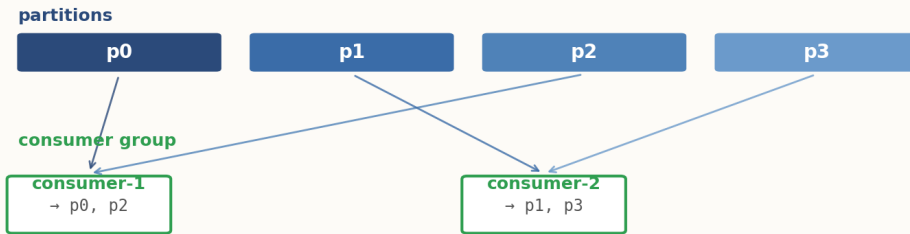
This chapter is about consumer groups, offsets, commits, and rebalances — the four words where almost all Kafka consumer bugs live.

Consumer groups

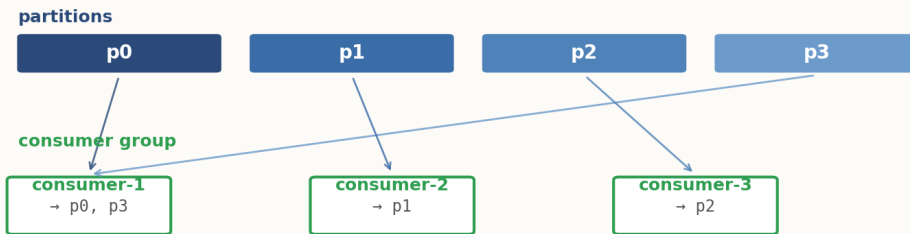
A single consumer process subscribing to a topic reads all partitions itself. Two consumer processes with the same `group.id` divide the partitions between themselves: each partition is assigned to exactly one member of the group.

Consumer group: parallelism caps at the partition count

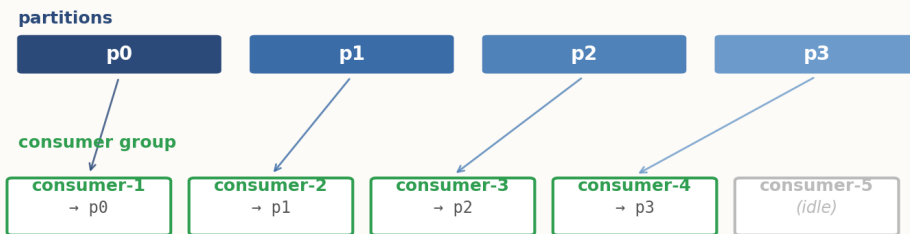
4 partitions, 2 members $\rightarrow 2 + 2$



4 partitions, 3 members $\rightarrow 2 + 1 + 1$



4 partitions, 5 members $\rightarrow 1 + 1 + 1 + 1 + \text{idle}$



A topic with 4 partitions served by a 2-member consumer group: partitions are split 2-and-2. Adding a third member gives 2/1/1; a fourth gives 1/1/1/1. Adding a fifth leaves one member idle.

Three rules fall out of that picture:

1. **Within a group, a partition has exactly one reader at a time.** Parallelism inside a group is capped at the partition count.
2. **Across groups, each group reads the full topic independently.** Adding an analytics pipeline next to your real-time pipeline costs disk reads, not consumer coordination.
3. **Scaling down to zero members means no one is reading.** The group's committed offset stays put; when a new member joins, it resumes from there.

```
from confluent_kafka import Consumer

c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id":         "orders-ui",
    "enable.auto.commit": False,      # commit manually — see below
    "auto.offset.reset": "earliest",  # only used when no offset exists
})
c.subscribe(["orders"])
```

The three positions a consumer tracks

Reading is a conversation between the consumer and the *group coordinator* (a broker). Three offsets are at play:

- **Fetch position.** What will `poll()` return next. Advances as you read.
- **Committed offset.** What the broker thinks you've remembered. Survives restarts.
- **Last processed offset.** What your application has actually acted on. Lives in your code, not in Kafka.

Consumer bugs almost always come from mixing these up. The common failure mode is: poll a record, commit its offset, then crash before processing the record. On restart, the consumer picks up after the committed offset — skipping the record that never got processed.

Warning

`enable.auto.commit=True` (the default) is the source of this bug. Auto-commit commits the *fetch* position on a timer, without any idea of whether you've finished processing. For anything beyond a demo, turn it off and commit explicitly.

Committing explicitly

The safe pattern is: poll → process → commit, in that order.

```
c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id":          "orders-ui",
    "enable.auto.commit": False,
    "auto.offset.reset": "earliest",
})
c.subscribe(["orders"])

try:
    while True:
        msg = c.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            log.error("poll error: %s", msg.error()); continue

        process(msg)                    # idempotent, please
        c.commit(message=msg, asynchronous=False)
finally:
    c.close()
```

Processing before committing gives you at-least-once semantics: a crash between `process(msg)` and `commit` replays the record on restart. That's fine, provided `process` is idempotent — writing to a database by primary key is idempotent, sending an email is not.

For throughput, commit in batches rather than per record:

```

batch = []
while True:
    msg = c.poll(1.0)
    if msg is None:
        if batch: commit_batch(c, batch); batch = []
        continue
    process(msg); batch.append(msg)
    if len(batch) >= 1000:
        commit_batch(c, batch); batch = []

def commit_batch(c, msgs):
    # Commit the highest offset per partition — that's what Kafka stores.
    from collections import defaultdict
    tops = defaultdict(lambda: -1)
    for m in msgs:
        tops[(m.topic(), m.partition())] = max(tops[(m.topic(),
                                                         m.partition())],
                                                m.offset())

    from confluent_kafka import TopicPartition
    c.commit(offsets=[TopicPartition(t, p, off + 1)
                      for (t, p), off in tops.items()],
              asynchronous=False)

```

The `+1` in that last line is the single most-debugged off-by-one in Kafka: the committed offset is the *next record to read*, not the last one processed.

Rebalances

When a group member joins or leaves (or gets kicked out for being slow), the coordinator redistributes partitions — a *rebalance*. During a rebalance, consumers stop reading, drop their in-flight batches, and re-fetch from the last committed offset.

Rebalances are the silent killer of consumer throughput. They are triggered by:

- A consumer crashing or being killed.
- A consumer failing to call `poll()` within `max.poll.interval.ms` (default 5 minutes). The coordinator kicks it out.
- Adding a new consumer to the group.
- Topic metadata changes (e.g. new partitions).

Tip

If your processing sometimes takes longer than 5 minutes per poll, *don't* crank `max.poll.interval.ms`. Instead, lower `max.poll.records` (or the equivalent byte-based setting) so each poll returns less work, and keep processing fast.

Modern Kafka supports *cooperative* rebalancing (via the `cooperative-sticky` partition assignor) that only reassigns partitions whose owner has changed, instead of the whole group pausing. Turn it on:

```
c = Consumer({
    # ...
    "partition.assignment.strategy": "cooperative-sticky",
})
```

`seek()`, and when to use it

Sometimes you want to ignore committed offsets and re-read records. Use `seek()`:

```
from confluent_kafka import TopicPartition

tps = c.assignment()
# Go to the beginning
c.seek(TopicPartition("orders", tps[0].partition, 0))
```

Or seek by timestamp:

```
tps = [TopicPartition("orders", p.partition, 1_713_600_000_000)
        for p in c.assignment()]
offsets = c.offsets_for_times(tps)
for tp in offsets:
    c.seek(tp)
```

`seek()` only takes effect for partitions currently assigned to you, so call it *after* the first successful `poll()` (which triggers the initial partition assignment) or use the `on_assign` callback on `subscribe`.

A resilient consumer skeleton

Pulling it together — a shape that survives a bad pod day:

```
from confluent_kafka import Consumer, KafkaException

def run():
    c = Consumer({
        "bootstrap.servers": "localhost:9092",
        "group.id": "orders-ui",
        "enable.auto.commit": False,
        "auto.offset.reset": "earliest",
        "partition.assignment.strategy": "cooperative-sticky",
        "max.poll.interval.ms": 300_000,
        "session.timeout.ms": 45_000,
    })
    c.subscribe(["orders"])

    try:
        while True:
            msg = c.poll(1.0)
            if msg is None: continue
            if msg.error():
                raise KafkaException(msg.error())
            try:
                process(msg)          # idempotent work here
            except Exception:
                log.exception("processing failed; will replay")
                continue             # do NOT commit
            c.commit(message=msg, asynchronous=False)
    finally:
        c.close()
```

Notice what's *not* here: retries inside the consumer, DLQ handling, circuit breakers. Those belong in `process()`, not in the consumer loop. Keep the loop boring.

Exercises

1. Run two consumer processes with the same `group.id` against a topic with 4 partitions. Kill one of them. How long until the other starts reading the abandoned partitions?
2. A consumer processes each record in ~7 minutes. What goes wrong? Which config change fixes it?

3. You need to re-run yesterday's data through a freshly-written consumer. Sketch two approaches: using a new `group.id`, or using `seek()` on an existing one. What are the trade-offs?
4. Explain in one sentence why the committed offset is "one past the last record processed" rather than the offset of the last record itself.

CHAPTER

Delivery Semantics

"Exactly once" is the phrase that sells Kafka talks and ships bugs. It is also, carefully defined, a real and achievable property of a Kafka pipeline. The trick is knowing what you are protecting against, what guarantees each moving part provides, and where the seams in the system are.

This chapter walks through the three classic delivery guarantees, shows how each one looks in Python, and explains the one guarantee Kafka genuinely gives you for free: per-partition *processing-order* when producers are idempotent and consumers commit properly.

The three semantics, concretely

Given: a producer sends a record, a consumer processes it.

Semantic

A record might be...

Processing is idempotent?

At most once

lost

not required

At least once

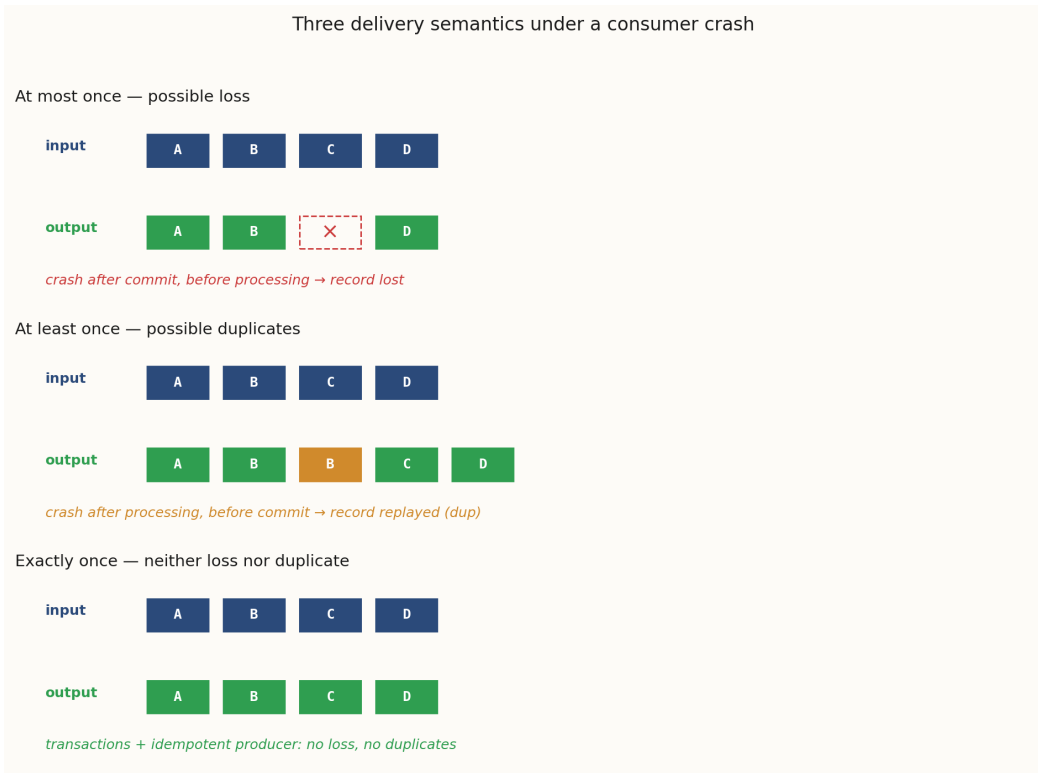
duplicated

required

Exactly once

neither

not required



Three semantics side-by-side: "at most once" loses records on failure, "at least once" duplicates, "exactly once" neither. The cost rises left to right.

- **At most once.** Send without retrying; consume and commit before processing. On any failure, the record is gone. Suitable for telemetry that you'd rather skip than double-count.
- **At least once.** Send with retries; consume, process, then commit. On failure, the record replays. Suitable when `process()` is idempotent.
- **Exactly once.** The broker, producer, and consumer cooperate so that a record is processed once and only once, even through failures. Suitable when dedup in the sink is expensive or impossible.

Most production pipelines are at-least-once with idempotent sinks. Exactly-once is a real feature, not a marketing phrase, but it has configuration cost — we'll see where.

Where duplicates actually come from

It helps to name the failure modes in one place:

1. **Producer retry.** Producer sends a batch; broker writes it; broker's ack is lost; producer retries; broker writes a duplicate. Fixed by `enable.idempotence=True`.
2. **Consumer commit-then-crash.** Consumer polls a record, commits offset, then crashes before processing. On restart, the record is skipped. Fixed by *never* committing before processing.
3. **Consumer process-then-crash.** Consumer polls a record, processes it, then crashes before committing. On restart, the record is re-processed. Fixed by making `process()` idempotent, or by using transactions.
4. **Rebalance mid-batch.** Consumer group rebalances while a batch is being processed; uncommitted records replay on the new owner. Same fix as (3).

Idempotent producers close (1). Commit-after-processing closes (2). Everything else requires either idempotency in your sink or the transactional APIs.

Idempotent producer (already on)

The shortest way to eliminate producer-side duplicates:

```
p = Producer({
    "bootstrap.servers": "localhost:9092",
    "enable.idempotence": True,    # producer IDs + sequence numbers
    "acks": "all",                # forced by idempotence anyway
})
```

That is the whole change. The broker now recognises the producer's ID and drops duplicate (producer-id, sequence-number) pairs. This is free from your point of view — cheap enough that there is no reason to leave it off.

Transactions: atomic write-and-commit

For true exactly-once across a consume-process-produce loop, Kafka offers transactions. A transaction groups several produces and one offset commit into a single atomic unit: either all of them land, or none do.

The canonical shape is a stream-processing worker: read from A, transform, write to B, commit the consumed offset of A — all transactionally.

```
from confluent_kafka import Producer, Consumer, TopicPartition

p = Producer({
    "bootstrap.servers": "localhost:9092",
    "enable.idempotence": True,
    "transactional.id": "etl-orders-v1",
})
p.init_transactions()

c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "etl-orders",
    "enable.auto.commit": False,
    "isolation.level": "read_committed", # see below
})
c.subscribe(["orders"])

while True:
    msg = c.poll(1.0)
    if msg is None or msg.error(): continue

    p.begin_transaction()
    try:
        out = transform(msg.value())
        p.produce("orders-enriched", key=msg.key(), value=out)

        # Include the consumed offset in the transaction
        p.send_offsets_to_transaction(
            [TopicPartition(msg.topic(), msg.partition(),
                           msg.offset() + 1)],
            c.consumer_group_metadata(),
        )
        p.commit_transaction()
    except Exception:
        p.abort_transaction()
        raise
```

Two knobs in this code deserve highlighting:

- `transactional.id` uniquely identifies this producer across restarts. It lets the broker fence zombie producers that a new instance has replaced.
- `isolation.level=read_committed` on the downstream consumer ensures it only sees records from committed transactions — never records from an in-flight or aborted transaction.

Warning

Transactions are not free. Every commit round-trips to the transaction coordinator; throughput per producer drops by 10–30%. Use transactions where you need them, not as a default.

Exactly-once in practice

The hard truth is that exactly-once is only end-to-end if your *external sinks* cooperate. Kafka transactions give you exactly-once between Kafka topics. When your consumer writes to PostgreSQL or S3, the guarantee ends at the consumer's last commit — the sink needs its own story.

Two ways to bridge the gap:

1. **Idempotent sink.** Design the write to PostgreSQL / S3 / whatever so that repeating it is a no-op. `INSERT ... ON CONFLICT DO NOTHING` with a stable key is the canonical example. Then at-least-once plus an idempotent sink is indistinguishable from exactly-once.
2. **Two-phase sink.** Write to the sink and commit the offset in one transaction (Debezium's outbox pattern, or a database that can enroll in the same transaction as Kafka). This is more work and requires sink-specific support.

Tip

When in doubt, design for at-least-once with idempotency. It is simpler, cheaper, and composes better across polyglot services than Kafka transactions do.

A worked comparison

A tiny program that writes to a transactional output topic, then crashes before committing. Re-run it: with transactions off, the output topic has duplicates; with transactions on, it doesn't:

```
def write_one(use_tx: bool):
    cfg = {"bootstrap.servers": "localhost:9092",
          "enable.idempotence": True}
    if use_tx: cfg["transactional.id"] = "demo-tx"
    p = Producer(cfg)

    if use_tx: p.init_transactions(); p.begin_transaction()
    p.produce("demo", key="k", value="v"); p.flush()
    if use_tx: p.commit_transaction()
```

Call `write_one(use_tx=False)` twice; the downstream consumer (with default `read_uncommitted`) sees two records. Call `write_one(True)` twice; the downstream consumer with `isolation.level=read_committed` still sees two — because each call was a committed transaction. But if the second call *aborts* (raise an exception before `commit_transaction`), that record never appears downstream.

Exercises

1. A payments service is at-least-once with a non-idempotent sink (a bank API). A retry causes a double charge. Which of the three semantic options from the table at the top genuinely fixes this? Which one does the team usually pick in practice?
2. Your transactional consumer sets `isolation.level=read_committed`. An upstream producer sends 1 million records in one transaction and then aborts. What does the consumer's `poll()` return during and after the abort?
3. A consumer reads from A and writes to B, idempotently, by primary key. It does *not* use Kafka transactions. Argue in four sentences whether this pipeline is exactly-once end-to-end.
4. Under what failure can idempotent-producer duplicates still slip through? (Hint: think about the producer PID across restarts of the process without a transactional id.)

CHAPTER

Schemas and the Registry

Kafka values are bytes. That is liberating for the broker and dangerous for everyone else: two services agree on what those bytes mean or they produce landmines for each other. JSON everywhere works until the day the producer starts sending a field as a string that used to be an integer, and an analytics job that has run fine for nine months quietly starts filtering out a third of the data.

A *schema* makes the contract between producer and consumer explicit. A *schema registry* stores schemas centrally and enforces evolution rules, so that a change shipped on Tuesday doesn't break a consumer team's release on Wednesday.

The three serialisation options

Three formats dominate Kafka deployments:

Format

Human-readable

Compact

Typed

Evolution story

JSON

yes

worst

loose

JSON Schema, ad-hoc

Avro

no

good

strict

built-in, well-defined

Protobuf

no

best

strict

via `reserved` fields

For most new projects in Python, **Avro + Confluent Schema Registry** is the path with the least friction. The Python client has first-class support; schema evolution is well-defined; the wire format is compact.

An Avro record, from Python

Install the extras and point a client at the registry:

```
pip install "confluent-kafka[avro]"
```

Define a schema in Avro IDL (a file on disk or a string in code):

```
{
  "type": "record",
  "name": "OrderPlaced",
  "namespace": "com.example.orders",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "user_id", "type": "string"},
    {"name": "amount", "type": "double"},
    {"name": "currency", "type": "string", "default": "USD"}
  ]
}
```

Produce with the Avro serialiser:

```
from confluent_kafka import Producer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import (SerializationContext,
                                           MessageField)

sr = SchemaRegistryClient({"url": "http://localhost:8081"})

with open("order_placed.avsc") as f:
    schema_str = f.read()

avro_ser = AvroSerializer(sr, schema_str)

p = Producer({"bootstrap.servers": "localhost:9092"})

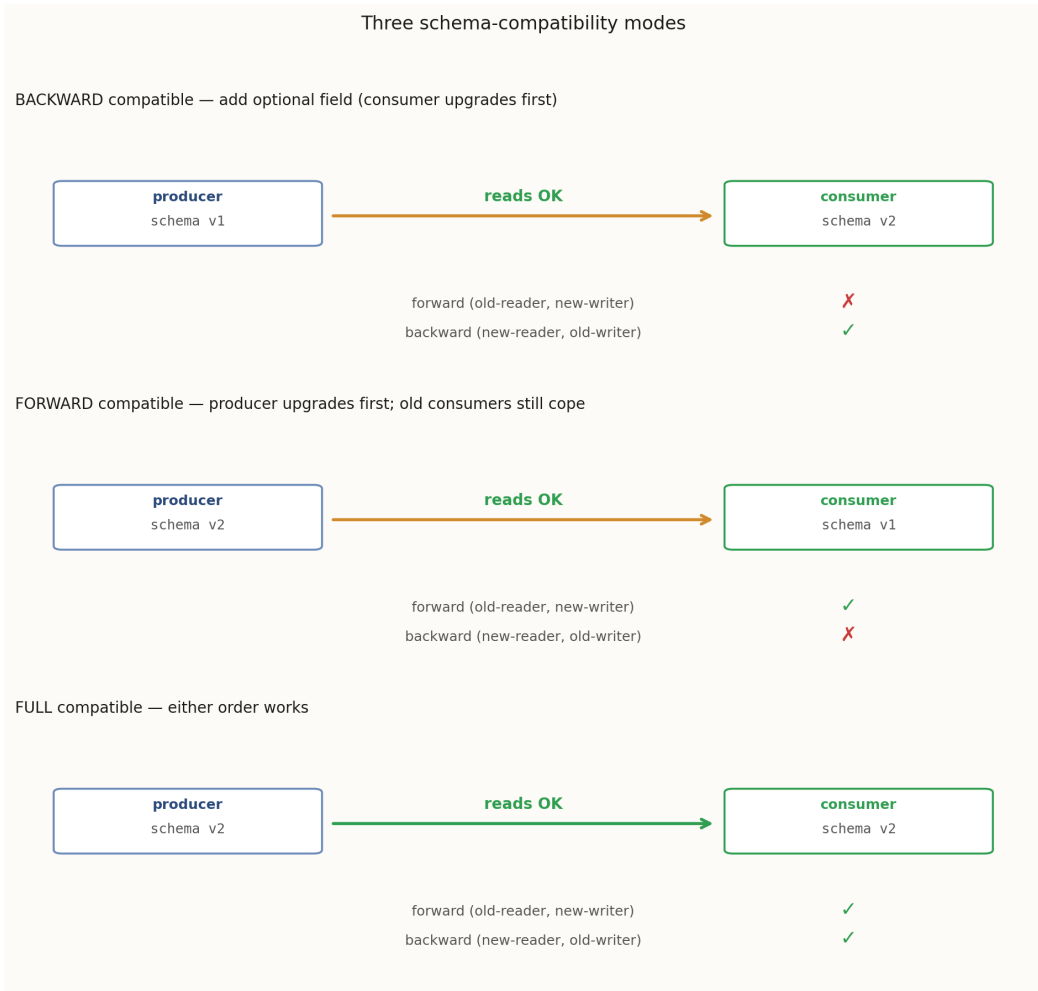
record = {"order_id": "42", "user_id": "u-1", "amount": 9.99,
          "currency": "USD"}
p.produce(
    "orders",
    key=record["order_id"],
    value=avro_ser(record, SerializationContext("orders",
                                                MessageField.VALUE)),
)
p.flush()
```

The first produce of a schema registers it with the registry and stamps the record with the returned *schema ID*. Consumers read the ID off the record, fetch the matching schema from the registry, and deserialise. The schema never travels on the wire — only a 4-byte ID — so the size overhead is minimal.

Schema evolution

The registry enforces a *compatibility level* per subject (most commonly one subject per topic). The important levels:

- **Backward** — new schema can read data written in the old schema. Good when consumers upgrade before producers. *The most useful default.*
- **Forward** — old schema can read data written in the new schema. Good when producers upgrade first and consumers lag.
- **Full** — both backward and forward. Safest, most restrictive.



Three evolution scenarios: backward-compatible (consumer upgrade first), forward-compatible (producer upgrade first), full (either order).

A *backward-compatible* change is typically:

- Adding a new field **with a default value**.
- Removing a field that has a default (so old readers can still populate it).
- Widening a numeric type, within carefully-specified limits.

A *breaking change* is:

- Renaming a field without aliases.

- Changing a field's type incompatibly (string → int).
- Adding a required field with no default.
- Removing a non-default field.

The registry will reject a breaking change on a subject with compatibility `BACKWARD`. That single policy, applied consistently, prevents an enormous class of production incidents.

Tip

Set `BACKWARD` compatibility on every topic that exists long enough to have consumers. The registry will tell you "no" before your broken schema hits production, and "no" is exactly what you want it to say.

A compatible evolution in practice

Version 1 of `OrderPlaced` is the schema above. Version 2 adds an optional `channel` field:

```
{
  "type": "record",
  "name": "OrderPlaced",
  "namespace": "com.example.orders",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "user_id", "type": "string"},
    {"name": "amount", "type": "double"},
    {"name": "currency", "type": "string", "default": "USD"},
    {"name": "channel", "type": ["null", "string"], "default": null}
  ]
}
```

Upgrade producer first (sends v2); upgrade consumer later (still reads with v1; sees `channel` default to `null`). Or: upgrade consumer first (reads v2; sees `channel = null` for records written by v1 producers). Either ordering works, because the change is backward- and forward-compatible — a *full-compatible* change.

The change that bites is subtle: renaming `amount` to `total`. Even if both old and new producers send "the same thing," Avro sees two differently-named fields. To rename, Avro lets you add an alias:

```
{"name": "total", "type": "double", "aliases": ["amount"]}
```

A reader using the new schema can read old records (the alias points back to `amount`). But old readers cannot read new records without falling back to a default — so this is backward-only. Ship it with a clear staging plan.

Schema-less pragmatism

Not every team has a schema registry in their stack. Some workable alternatives, roughly in order of rigour:

1. **JSON + JSON Schema files in a shared repo.** Producers and consumers both load the schema at build time; CI validates. Works; no central enforcement, so it only helps disciplined teams.
2. **Versioned topic names.** `orders-v1`, `orders-v2`. A breaking change becomes a new topic; old consumers keep working; new consumers read the new topic. No registry needed, but you end up with consumer code that understands multiple versions of the world.
3. **Protobuf with a vendored .proto.** Protobuf's own compatibility rules (never remove tag numbers, reserve removed fields, add only `optional` fields) give you a lot of what a registry does, by convention.

Pick one and stick with it. The worst option is "we all use JSON and we'll remember."

Exercises

1. You add a required `tenant_id` field to an Avro schema with `BACKWARD` compatibility. The registry rejects the change. Why? What is the smallest fix?
2. A producer in version 2 sends a new enum value that version 1 does not know about. Version 1 consumers start failing. Which compatibility setting would have caught this at produce time?
3. Rewrite the Avro evolution rules in your own words, in three sentences, so that a teammate who hasn't read this chapter can answer: "is adding an optional field safe?"
4. A team uses JSON without a registry. They want to safely rename `amount` to `total`. Sketch a rollout plan that avoids dropping events.

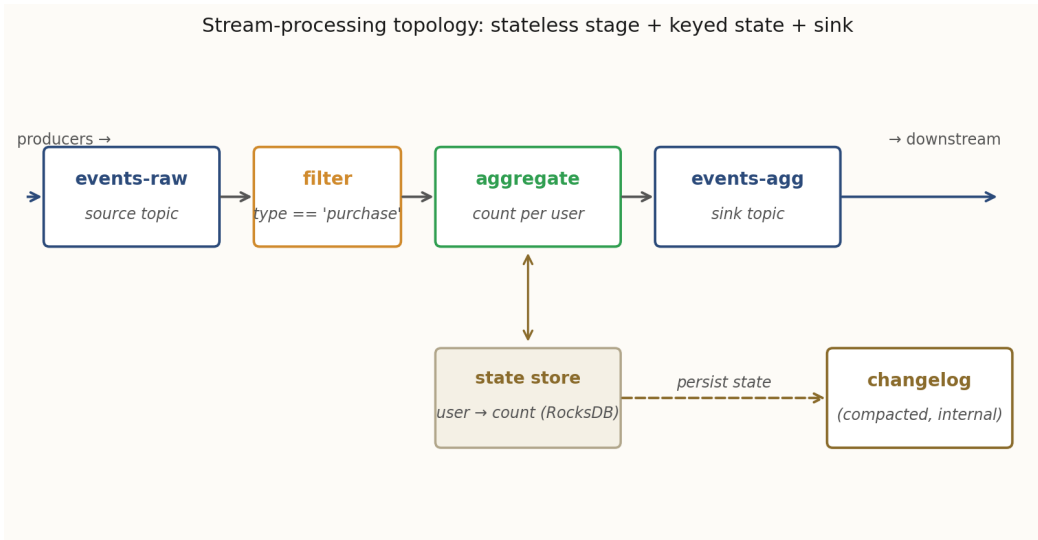
Stream Processing

So far every consumer we have written has been a loop that pulls records and does something with each one. That model covers a lot of ground — it covers every chapter before this one — but there are classes of problem it doesn't touch: joining two streams, aggregating by key with windowing, maintaining materialised state across restarts. This is stream processing.

Kafka's own stream-processing library is *Kafka Streams*, which is Java-only. Python has three practical alternatives: **Faust** / **Faust-streaming**, **Bytewax**, and the lower-level approach of **writing the state yourself**. Each is a different trade-off between ergonomics and operational simplicity.

What stream processing actually adds

Batch processing reads a bounded input and produces a bounded output. Stream processing reads an unbounded input and produces an unbounded output — but the interesting part is the *state* that lives between records. A running total, a sessionisation window, the current state of an order.



A typical streaming topology: input topic → filter → keyed aggregation → output topic, with a state store persisted as a compacted changelog alongside.

Four shapes of streaming operation show up again and again:

- **Stateless transformations.** `map`, `filter`, `flatMap`. One record in, zero or more out. No state.
- **Keyed aggregations.** Count, sum, min/max by key. State grows with the number of distinct keys.
- **Windowed aggregations.** "Sessions per user per 5-minute window." State grows with (keys × active windows).
- **Stream-stream joins.** "Match a click to an impression within 30 seconds." State is a per-key buffer with a time-based retention.

The last three all need durable state that survives a process restart. The clever trick in modern stream processors is to publish every state change to a compacted Kafka topic (a *changelog*) so that on restart the state store can be rebuilt by re-reading the changelog.

Approach 1: Faust-streaming

Faust is the Python port of Kafka Streams. The original project stalled; the community fork `faust-streaming` is actively maintained and works with modern Kafka versions.

```
pip install faust-streaming
```

A minimal Faust app that counts orders per user over a 1-minute tumbling window:

```
import faust

app = faust.App("orders-count",
                broker="kafka://localhost:9092",
                store="rocksdb://")

class Order(faust.Record, serializer="json"):
    order_id: str
    user_id: str
    amount: float

orders = app.topic("orders", value_type=Order)

counts = (app.Table("user_minute_counts", default=int)
          .tumbling(60.0, expires=3600.0))

@app.agent(orders)
async def process(stream):
    async for order in stream.group_by(Order.user_id):
        counts[order.user_id] += 1

if __name__ == "__main__":
    app.main()
```

Faust handles the part that is hard to write yourself: partition assignment, rebalancing, changelog topics for `counts`, restoration from the changelog after a crash. State lives in RocksDB on local disk; Kafka stores the changelog.

Note

Faust has real warts — async-heavy API, opinionated startup, slower than JVM equivalents — but it is by far the quickest path to "a real stream processor in Python without standing up a new infrastructure piece."

Approach 2: Bytewax

Bytewax is a newer alternative that compiles Python dataflows onto a Rust-based runtime. It trades Faust's table abstraction for a simpler "operators on streams" model and is notably faster on CPU-heavy transformations.

```
pip install bytewax
```

```
from bytewax.dataflow import Dataflow
from bytewax.connectors.kafka import (KafkaSource, KafkaSink,
                                      KafkaSourceMessage)
from bytewax import operators as op

flow = Dataflow("orders-enrich")
inp = op.input("in", flow, KafkaSource(["localhost:9092"], ["orders"]))

parsed = op.map("parse", inp, lambda m: json.loads(m.value))
high = op.filter("high", parsed, lambda o: o["amount"] > 100)
out = op.map("to-kafka", high,
            lambda o: KafkaSourceMessage(key=o["order_id"].encode(),
                                          value=json.dumps(o).encode()))
op.output("sink", out, KafkaSink(["localhost:9092"], "orders-high"))
```

Bytewax supports windowed state, joins, and has a cleaner deployment story (single binary per worker) than Faust. If you're starting a new Python streaming job today, it's the option I'd reach for first.

Approach 3: do it yourself

For simple keyed aggregations over short windows, you don't always need a framework. A vanilla consumer plus a dict plus a commit strategy will do:

```
from collections import defaultdict
from confluent_kafka import Consumer

c = Consumer({"bootstrap.servers": "localhost:9092",
             "group.id": "manual-counter",
             "enable.auto.commit": False,
             "auto.offset.reset": "earliest"})
c.subscribe(["orders"])

counts = defaultdict(int)
last_flush = time.time()

while True:
    msg = c.poll(1.0)
    if msg and not msg.error():
        order = json.loads(msg.value())
        counts[order["user_id"]] += 1

    if time.time() - last_flush > 60:
        publish(counts)          # push to Kafka or a DB
        c.commit(asynchronous=False)
        counts.clear()
        last_flush = time.time()
```

This works and is often the right shape for "I just need to count things." The moment you need fault-tolerant state (surviving a pod restart without losing in-flight counts), you're on the path towards reinventing a framework; stop and pick Faust or Bytewax instead.

When *not* to reach for a streaming framework

Three anti-patterns worth naming:

- **You just need to filter.** A one-line `filter` in a plain consumer is clearer than any framework.
- **You have a single cross-record aggregation.** If a running sum or count is the whole job, a consumer + a database is simpler.
- **Your data is bounded.** A file dropping into S3 every hour is a batch job. Kafka + streaming doesn't automatically help.

Tip

The rule of thumb: reach for a streaming framework when your job has both **keyed state** and **fault tolerance** requirements. One without the other usually fits in a 50-line consumer.

Exercises

1. In 100 lines of Python, build a stream processor that computes the running mean of `amount` per `user_id`, publishing updates every second to an output topic. Use the plain consumer approach.
2. Port the same logic to Faust. Which lines of your code disappeared? Which operational properties did Faust give you for free?
3. An analyst asks for "the top 10 users by spend in the last hour, updated in real time." Sketch the topology — which operators, what state, what windowing.
4. A windowed aggregation keeps 60 minutes of state across 100k distinct keys. Estimate the RocksDB footprint. (Hint: make an assumption about bytes-per-key and multiply.)

CHAPTER

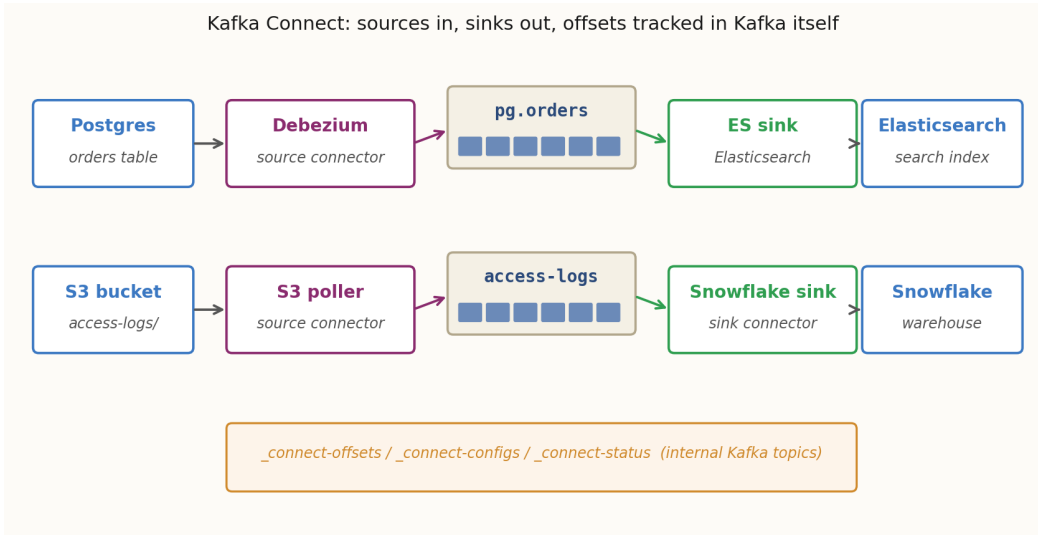
Kafka Connect

A lot of Kafka work is pushing data between Kafka and a system that isn't Kafka: a database, a blob store, a search index, a warehouse. You can write a custom producer or consumer for each pair, and often people do, but after the fourth time you write "read a Postgres changelog and write it to a topic" you start to wonder if someone has solved this.

They have. *Kafka Connect* is a process that runs connector plugins — source connectors that pull from external systems into Kafka, and sink connectors that push from Kafka out. It handles the boring parts: fault tolerance, offset tracking, parallel tasks, schema handling. Your job is to pick the right connector and configure it.

The shape of Connect

Connect runs as its own JVM process (or cluster), separate from the brokers. It loads connector plugins from a classpath, exposes a REST API for configuration, and coordinates tasks through Kafka itself.



Connect cluster with two source connectors (Postgres CDC, S3 file poller) writing into Kafka, and two sink connectors (ElasticSearch, Snowflake) reading out. Offsets and connector state live in internal Kafka topics.

A *connector* is a deployment unit. It splits into one or more *tasks*, each of which is effectively a partition-bound consumer (for sinks) or an independent reader (for sources). Connect auto-rebalances tasks across workers when a worker joins or leaves.

Connect is intentionally JVM-only. You run it; you don't write it in Python. Python's role is to (a) configure connectors through the REST API, (b) consume the topics Connect produces into, and (c) produce to topics that Connect drains out.

Running Connect locally

The easiest local Connect is the Confluent Platform image, or Strimzi's community Connect image. With docker-compose, adding a worker next to your broker is four lines:

```
connect:
  image: confluentinc/cp-kafka-connect:7.6.0
  depends_on: [kafka]
  ports: ["8083:8083"]
  environment:
    CONNECT_BOOTSTRAP_SERVERS: kafka:9092
    CONNECT_REST_ADVERTISED_HOST_NAME: connect
    CONNECT_GROUP_ID: connect-cluster
    CONNECT_CONFIG_STORAGE_TOPIC: _connect-configs
    CONNECT_OFFSET_STORAGE_TOPIC: _connect-offsets
    CONNECT_STATUS_STORAGE_TOPIC: _connect-status
    CONNECT_KEY_CONVERTER: org.apache.kafka.connect.json.JsonConverter
    CONNECT_VALUE_CONVERTER: org.apache.kafka.connect.json.JsonConverter
    CONNECT_PLUGIN_PATH: /usr/share/java
```

Health-check it:

```
$ curl -s http://localhost:8083/ | jq
{
  "version": "7.6.0-ccs",
  "commit": "...",
  "kafka_cluster_id": "..."
}
```

List available connectors:

```
curl -s http://localhost:8083/connector-plugins | jq -r '.[].class'
```

Source connector example: a file

The built-in `FileStreamSource` is the "hello world" of Connect: it tails a file and emits each line as a record. Useful for nothing in production, good for wiring up your first connector:

```
import requests

requests.post("http://localhost:8083/connectors",
              json={
                  "name": "tail-access-log",
                  "config": {
                      "connector.class":
                          "org.apache.kafka.connect.file.FileStreamSource",
                      "tasks.max": "1",
                      "file": "/tmp/access.log",
                      "topic": "access-log",
                  },
              })
```

Anything appended to `/tmp/access.log` now turns into records on the `access-log` topic. Connect tracks the file offset it last read in `_connect-offsets`, so a worker restart resumes where it left off.

Sink connector example: S3

A more realistic shape — dumping a topic into S3 as time-partitioned Parquet files, for downstream batch analysis:

```
requests.post("http://localhost:8083/connectors", json={
    "name": "orders-to-s3",
    "config": {
        "connector.class":
            "io.confluent.connect.s3.S3SinkConnector",
        "tasks.max": "4",

        "topics": "orders",
        "s3.bucket.name": "my-data-lake",
        "s3.region": "us-west-2",

        "format.class":
            "io.confluent.connect.s3.format.parquet.ParquetFormat",
        "partitioner.class":
            "io.confluent.connect.storage.partitioners."
            "TimeBasedPartitioner",
        "path.format": "YYYY/MM/dd/HH",
        "flush.size": "10000",
        "rotate.interval.ms": "600000",
    },
})
```

Four tasks, which means four partitions of `orders` are read in parallel. Every 10,000 records or 10 minutes — whichever first — the task writes a file. S3 prefixes partition by the hour so downstream queries can prune.

CDC: the killer use case

Connect's most valuable use is *change data capture*: streaming a relational database's write-ahead log as topic records. **Debezium** is the most popular implementation. A single connector turns every `INSERT / UPDATE / DELETE` on a Postgres table into a Kafka record, keyed by primary key, with both the old and new row.

```
requests.post("http://localhost:8083/connectors", json={
    "name": "pg-orders-cdc",
    "config": {
        "connector.class":
            "io.debezium.connector.postgresql.PostgresConnector",
        "database.hostname": "pg",
        "database.port": "5432",
        "database.user": "debezium",
        "database.password": "****",
        "database.dbname": "app",
        "database.server.name": "app-prod",
        "table.include.list": "public.orders,public.customers",
        "plugin.name": "pgoutput",
        "topic.prefix": "pg",
    },
})
```

The `orders` table now streams into `pg.public.orders`. Every row change produces a record with a stable structure (`{before, after, op, source}`) and a key equal to the primary key. Point your microservices at the topic; they stop polling the database.

Tip

CDC + compacted topics is one of the most useful patterns in a modern data platform. Chapter 13 revisits it under "outbox pattern" and the related trade-offs.

When *not* to use Connect

- **The source/sink is custom and there's no existing connector.** Writing a new connector is non-trivial Java. For one-off integrations, a plain Python producer/consumer is faster.
- **You need to transform data in flight.** Connect's Single Message Transforms are limited. For anything beyond dropping a field or routing a topic, put a stream processor between Connect and the consumer.
- **The volume is tiny.** Connect is a whole JVM cluster. If you are moving 1 MB/day, a cron job is fine.

Monitoring Connect

The REST API is your window into a running worker:

```
# list connectors and their current state
curl -s http://localhost:8083/connectors?expand=status | jq .

# restart a failed task
curl -X POST http://localhost:8083/connectors/orders-to-s3/tasks/0/restart

# look at the last error
curl -s http://localhost:8083/connectors/orders-to-s3/status | jq \
  '.tasks[] | select(.state=="FAILED") | .trace'
```

In production, pair this with Prometheus JMX exporter scraping the Connect worker. The two metrics that catch most problems: `source-record-write-total` (is the source reading?) and `sink-record-send-total` (is the sink writing?). Flat lines on either are the first sign that a connector is stuck.

Exercises

1. Run a `FileStreamSource` pointed at `/tmp/events.log`, then a `FileStreamSink` reading from the same topic into `/tmp/events.out`. Confirm round-trip. Where does Connect store the file-read offset across restarts?
2. Given a Postgres table `orders(id, user_id, total, updated_at)`, write the Debezium config that captures changes into `pg.public.orders`. What would you key the topic on?
3. A sink connector lags its source by 2 hours. List four diagnostic steps in order, from the cheapest to the most invasive.
4. You need to join a CDC stream of `orders` with a static `products` lookup. Where does the join belong: in Connect, in a stream processor, in the downstream consumer?

CHAPTER

Replication and Durability

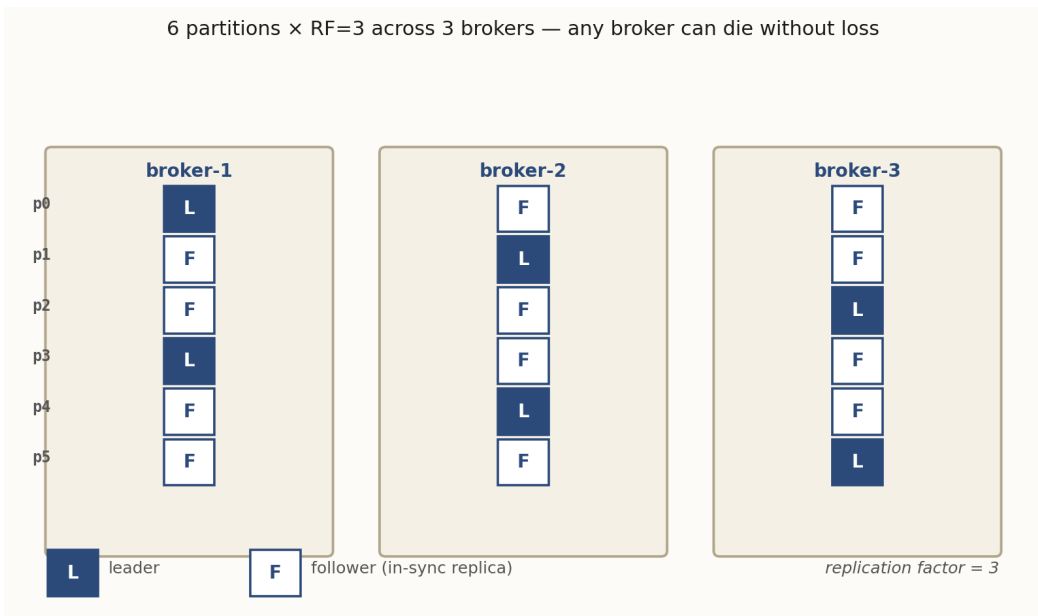
The single-broker setup in chapter 3 was fine for experimentation, but it offered no protection against hardware failure. A dead disk loses your topic. Production Kafka is a

cluster of several brokers that replicate partitions to each other so that no single failure ends the story.

This chapter walks through how Kafka replicates, what the in-sync-replica set is, how leader election works, and what configuration knobs turn "replicated" into "actually durable."

Replicas, leaders, followers

Every partition has a *replication factor* — the number of copies of its log that exist in the cluster. Replicas sit on different brokers. One replica is the *leader*; the others are *followers*. All client traffic (produces, fetches) goes through the leader; followers asynchronously fetch from the leader and keep their copy up to date.



A topic with 6 partitions and replication factor 3 spread across 3 brokers. Each partition has one leader (solid) and two followers (hollow). A broker crash leaves two replicas of each partition alive; a new leader is elected from those.

A partition's replica set is static (you pick it at topic-create time and only reassignment can change it). What moves around is leadership: if broker 2 crashes, its leaderships migrate to followers on surviving brokers. When broker 2 rejoins, followers catch up and the cluster returns to a steady state.

Note

Kafka does not "replicate a topic." It replicates each *partition*. Two partitions of the same topic can have entirely different replica sets. The controller's job is to keep this mapping balanced across brokers.

The ISR (in-sync replica set)

For each partition, the broker tracks which replicas are *in sync* with the leader. A follower is in sync if it has fetched records up to within `replica.lag.time.max.ms` (default 30 seconds) of the leader's log-end. Lagging followers drop out of the ISR; once they catch up, they rejoin.

Two configuration knobs determine durability:

- `replication.factor` (at topic create time): how many copies exist.
- `min.insync.replicas` (at topic or broker level): the minimum number of in-sync replicas required for a produce to succeed when `acks=all`.

A common and sensible setup is `replication.factor=3`, `min.insync.replicas=2`, `acks=all`. Interpretation: "the record must be on two of three broker disks before the producer sees an ack." Kafka tolerates one broker down (ISR shrinks to 2, still meets `min.insync.replicas`) without data loss. A second broker failure causes produces to fail loudly — which is usually what you want, because otherwise you'd be writing to a single surviving replica and risking silent loss.

Warning

`replication.factor=3` alone is not enough. If `min.insync.replicas=1` and the leader is the only in-sync replica, a produce succeeds, then the leader dies, then a follower that didn't get the record is elected — the record is gone. Always set `min.insync.replicas` to at least 2 for RF=3.

The high-water mark, revisited

Remember from chapter 2: the *high-water mark* (HWM) is the latest offset that every in-sync replica has acknowledged. Consumers can only read up to the HWM. This is how

Kafka protects readers from eating records that might later be erased by a leader failover:

- Leader writes offset 100 to its log.
- Followers haven't fetched 100 yet. HWM stays at 99.
- Consumer fetches; it gets up to offset 99.
- Leader dies. New leader is a follower whose log ends at 99.
- The record at offset 100 on the old leader is effectively lost. Because it was never above HWM, no consumer saw it.

Producers with `acks=all` see this same boundary: an ack is only returned once the record is above HWM on the new leader's view of history, not just on the old leader.

Unclean leader election

If every replica of a partition is offline — say all three brokers went down simultaneously — Kafka by default refuses to elect a leader from an out-of-sync replica. That preserves durability but leaves the partition unavailable until an in-sync replica comes back.

`unclean.leader.election.enable=true` flips this: a follower that is out of sync *can* be elected leader, at the cost of losing any records that were on the failed leader but hadn't replicated yet. Useful for "availability trumps consistency" workloads (metrics, telemetry), dangerous for anything with business meaning.

The default is off, and the default is right for almost everyone.

Rack awareness

In a multi-rack or multi-AZ deployment, spreading replicas of a partition across racks protects against a whole rack / AZ dying at once. Kafka will honour `broker.rack` when placing partitions:

```
# in broker config
broker.rack=us-west-2a
```

With three brokers tagged `us-west-2a/b/c` and a topic with RF=3, the controller places one replica per rack. A single-AZ outage downgrades every partition from RF=3 to RF=2, but no partition loses all replicas.

Tip

Rack awareness is the cheapest and most underused durability improvement in Kafka. One config per broker, no runtime cost.

Reassignment: moving partitions

Sometimes a broker is overloaded, or you've added new brokers and want to rebalance, or you've decommissioned a broker and need to move its partitions away. Kafka's reassignment tool generates and applies a new replica placement:

```
# Write a JSON file listing topics to reassign
cat > reassign.json <<EOF
{"topics": [{"topic": "orders"}], "version": 1}
EOF

# Ask the tool to propose a new placement
kafka-reassign-partitions.sh --bootstrap-server localhost:9092 \
  --topics-to-move-json-file reassign.json \
  --broker-list "1,2,3,4" \
  --generate

# Save the proposed placement to proposed.json and apply
kafka-reassign-partitions.sh --bootstrap-server localhost:9092 \
  --reassignment-json-file proposed.json \
  --execute
```

Reassignment can saturate disk and network during the catch-up phase. Throttle it (`--throttle 10000000` for 10 MB/s) or you will cause a performance incident while trying to fix a performance problem.

Durability at the broker

Two last knobs to know about:

- `flush.messages` / `flush.ms` : how often the broker forces `fsync` of the log segment to disk. Defaults are effectively "never; let the OS decide," because Kafka relies on replication for durability rather than `fsync` . This is correct for a replicated cluster; on an unreplicated single-broker development box, consider setting a lower `flush.ms` .
- `log.dirs` : which directories hold log data. Pointing these at multiple independent disks gives you per-disk parallelism; losing one disk still loses the partitions on it, so let replication do its job.

A durability checklist

Before calling a production deployment "durable":

- `[] replication.factor >= 3` on every business-critical topic
- `[] min.insync.replicas >= 2`
- `[] acks=all` on every producer
- `[] enable.idempotence=true` on every producer
- `[] unclean.leader.election.enable=false` (default)
- `[] broker.rack` set on every broker
- `[]` Reassignment throttled in runbooks
- `[]` Monitoring on `UnderReplicatedPartitions` and `OfflinePartitionsCount`

None of these individually "make Kafka durable." Together they make a cluster where a single broker, disk, or rack can die without losing records that producers were told had been accepted.

Exercises

1. A cluster has `RF=3` and `min.insync.replicas=2` . Two brokers die. What happens to producers with `acks=all` ? To consumers?
2. You set `min.insync.replicas=3` on an `RF=3` topic. How does that change availability? Under what condition would any broker restart cause producer failures?

3. Under `unclean.leader.election.enable=true`, describe a concrete sequence of events that causes a consumer to first read record X and then have X disappear from later reads.
4. Your reassignment is saturating the cluster's network. What is the safe way to slow it down without aborting it?

CHAPTER

Performance and Tuning

Kafka is fast out of the box. The defaults are reasonable for most small-to-medium workloads, and the pathological-slow-Kafka story almost always traces to a misconfiguration in the client rather than in the broker. But "fast" is not a single number: Kafka can be latency-bound, throughput-bound, or CPU-bound depending on what you ask of it, and the knobs that help each case are different.

This chapter walks through where Kafka's time actually goes, what you should measure before tuning, and the handful of settings that move the needle.

Four numbers that matter

When someone says "Kafka is slow," it is worth a minute to pin down which number:

- **End-to-end latency.** Time from `produce()` to `consumer.poll()` returning the record. Usually 5–50 ms. Bounded by `linger.ms` on the producer and by poll frequency / replication on the broker.
- **Producer throughput.** Records or MB/s one producer can send. Bounded by batching, compression, network, and broker `fsync`.
- **Consumer throughput.** Records or MB/s one consumer can process. Bounded by `fetch.min.bytes`, `fetch.max.bytes`, and your own `process()` function.
- **Broker disk I/O.** MB/s the broker can land on disk without falling behind. Bounded by disk hardware, log segment size, and replication.

None of these caps each other automatically; optimising one can pessimise another. Know which you care about.

The first rule: measure

Before changing a setting, know what the current number is. For a quick throughput sanity check, Kafka ships `kafka-producer-perf-test` and `kafka-consumer-perf-test` :

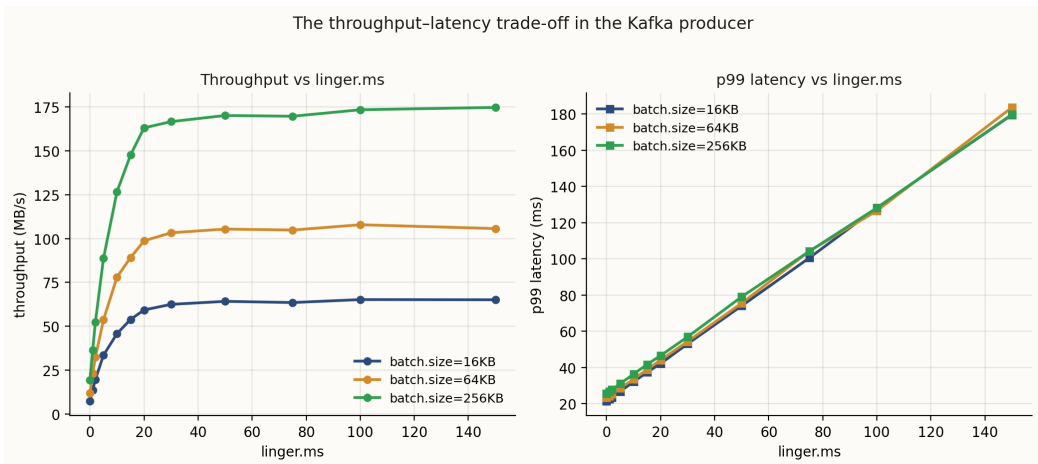
```
kafka-producer-perf-test.sh --topic bench \
  --num-records 1000000 --record-size 1024 \
  --throughput -1 \
  --producer-props bootstrap.servers=localhost:9092 \
    acks=all linger.ms=10 compression.type=zstd
```

Output:

```
100000 records sent, 52083 records/sec (50.85 MB/sec),
 45.3 ms avg latency, 210.0 ms max latency.
```

Repeat with your own settings. Two data points is enough to know whether a change helped.

The throughput–latency knobs



Producer throughput vs. linger.ms: a little linger buys big throughput gains by batching, flattening out around 10-20ms. Past that, you add latency with no throughput benefit.

Three producer settings dominate the throughput-vs-latency trade-off:

- `linger.ms` . How long to wait before flushing a partial batch. The default 0 means "flush every record immediately," which kills throughput. Setting 10–20 ms recovers most of the available throughput at a latency cost below human perception. Raising past 50 ms rarely helps.
- `batch.size` . Maximum batch size before an automatic flush. The default (16 KB) is small for high-throughput workloads; 64 KB or 128 KB pair well with compression.
- `compression.type` . `zstd` for best ratio; `lz4` for lowest CPU; `none` for the network-is-free case. Compression happens on batches, so a batch of 100 tiny records often compresses 4–8x.

On the consumer side:

- `fetch.min.bytes` . Minimum bytes to return from a fetch. A tiny default means lots of small fetches; setting 64 KB means the consumer waits briefly for more data. Pairs with `fetch.max.wait.ms` .
- `max.poll.records` . How many records each `poll()` returns. Large values reduce poll overhead but increase processing burst size.

Tip

The default settings favour latency. If you are doing a bulk backfill or batch ETL and latency doesn't matter, bump `linger.ms=50` , `batch.size=131072` , `fetch.min.bytes=65536` and watch throughput multiply. Restore defaults for the live path.

Partition count

We touched this in chapter 2. The rule of thumb for throughput:

- Each partition sustains roughly **a few MB/s to tens of MB/s** per producer on modern hardware, depending on batching.
- Each partition sustains roughly **tens of MB/s** per consumer, depending on `process()` cost.

If your topic needs 100 MB/s of producer throughput and one producer partition handles 20 MB/s, you need at least 5 partitions. Round up for headroom and for future consumer parallelism.

The ceiling is set by how many partitions a broker can comfortably lead. A modern broker handles a few thousand leaderships; past that, controller operations slow down. Aim for a couple hundred partitions per broker as a design target.

Broker-side

Three broker configs affect throughput directly:

- `num.network.threads` and `num.io.threads`. Network threads handle socket I/O; I/O threads handle disk. Defaults (3 and 8) are fine for small clusters; raise for dedicated broker hosts with many cores.
- `log.segment.bytes`. Size of each segment file. Default 1 GB. Larger segments mean fewer file handles; smaller segments mean retention deletions happen at finer granularity. The default is rarely the wrong answer.
- `log.retention.hours` / `.bytes`. How long to keep data. Not a performance knob directly, but over-long retention forces larger log directories and slows broker startup (scanning segments on boot).

For disks: prefer a single big dedicated disk per broker over many small ones. Sequential write bandwidth is what Kafka loves. NVMe is overkill for most workloads; a good SAS SSD is plenty.

The memory story

Kafka does not really cache records in JVM heap. It relies on the Linux page cache: recent segments stay in memory because the OS hasn't evicted them yet, and hot consumers read from cache rather than disk. The implication for sizing:

- JVM heap doesn't need to be huge. 6–8 GB is plenty for a broker doing normal work.
- **Most of the memory on a broker host should be free for page cache.** A 64 GB broker with 6 GB heap leaves 58 GB for the OS, and that is the right ratio.

Warning

A common anti-pattern is setting the JVM heap to 30+ GB "because we have a big server." This *shrinks* available page cache, *increases* GC pauses, and makes Kafka noticeably slower. Don't do it.

What a running broker looks like

When Kafka is healthy:

- Producer median latency < 20 ms, p99 < 100 ms.
- `UnderReplicatedPartitions = 0` in steady state.
- Network bytes in/out roughly balanced with disk bytes written.
- CPU well under 70% per core.

When it isn't, the first five metrics to check:

1. `UnderReplicatedPartitions` — replication is falling behind.
2. `RequestQueueTimeMs p99` — requests are queuing at the broker.
3. `RequestHandlerAvgIdlePercent` — is IO-thread pool saturated?
4. `LogFlushRateAndTimeMs` — disk has slowed down.
5. `Records{In, Out}PerSec` — do they match the app layer's view?

If you only instrument five things about Kafka, those are the five.

Exercises

1. A producer sends 10 KB records at `linger.ms=0`, `batch.size=16384`. Measured throughput is 3 MB/s. What two changes would you try first? Predict the effect of each.
2. A topic with 4 partitions is consumed by a group with 6 members. What is the maximum achievable consumer parallelism? What change would unlock more?
3. A broker runs with 128 GB of RAM and a 60 GB JVM heap. Describe the likely symptoms during a replication catch-up burst. What's the fix?
4. The producer client has `linger.ms=20`, but p99 latency is 500 ms. Where in the chain (producer batch, broker handling, replication, consumer poll) is the time most likely spent? How would you confirm it?

CHAPTER

Operating a Cluster

There is a version of this book that skips this chapter: the one where you use a managed Kafka (Confluent Cloud, MSK, Aiven) and someone else handles broker rotation, JVM upgrades, and disk capacity planning. If that's you, feel free to skim. The rest of you are running your own cluster, and there is a short list of things you should know how to do.

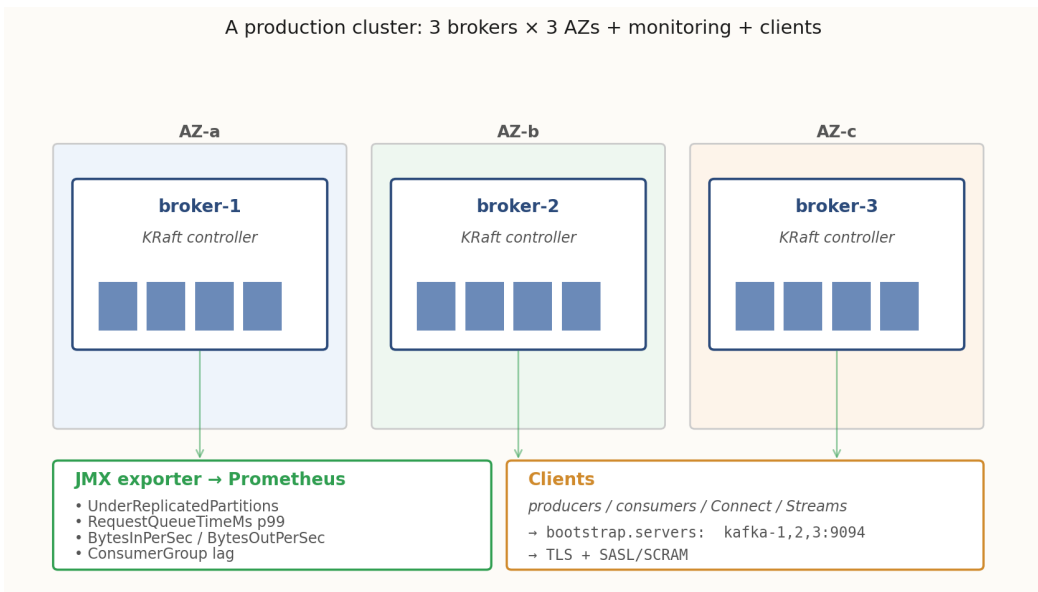
This chapter is that list: deploy, upgrade, secure, monitor.

Cluster shape

A production Kafka cluster has at least three brokers. Three is a magic number:

- A topic with `RF=3` can survive a single broker failure without loss.
- `min.insync.replicas=2` gives a producer a meaningful ack even with one broker down.
- Three is also the minimum quorum for KRaft controllers.

Smaller clusters work technically but leave you nowhere to stand when something breaks.



A typical production cluster: 3 brokers across 3 AZs, each broker also serving as a KRaft controller; a monitoring stack pulls JMX; clients reach brokers via a per-listener endpoint.

KRaft (Kafka's replacement for ZooKeeper, now default since 3.5) collapses the control plane into the brokers themselves. You no longer run a separate ZooKeeper ensemble; a

subset of brokers run a Raft group that manages cluster metadata. The operational savings are real: one less service to monitor, upgrade, and secure.

Deploying: the three realistic paths

- **Containers + Strimzi on Kubernetes.** The dominant shape for self-hosted Kafka in 2026. Strimzi gives you custom resources for clusters, topics, and users. Operators handle rolling upgrades and rebalances.
- **Confluent Platform on VMs with Ansible.** Conservative, well-documented, works. Preferred by teams that prefer immutable VMs to Kubernetes.
- **Managed.** Confluent Cloud, AWS MSK, Aiven. You lose some flexibility on tuning but skip most of this chapter.

Avoid writing bespoke deployment scripts from scratch. Kafka has enough operational nuance that a community-maintained deployer will be ahead of you.

Upgrades

Kafka's version compatibility is excellent: clients and brokers of adjacent minor versions interoperate. The canonical rolling upgrade pattern is:

1. Upgrade the inter-broker protocol version to the current (pre- upgrade) value in `server.properties` : `inter.broker.protocol.version=3.7`
2. Roll each broker to the new binary, one at a time. Wait for ISR to recover before proceeding to the next.
3. Once all brokers are on the new binary, bump `inter.broker.protocol.version` to the new value (`3.8`) and roll again.

The two-phase approach prevents a half-upgraded cluster from using protocol features that only some brokers speak.

Tip

Strimzi and Confluent's Ansible playbooks automate this whole dance. Manual upgrades are error-prone; prefer the tooling.

Security, briefly

Three concerns, three mechanisms:

- **Encryption in transit.** TLS on every listener. Modern Kafka makes this a 5-line config change; "plaintext in production" is no longer defensible.
- **Authentication.** SASL/SCRAM for most deployments, mTLS for strict environments. Avoid SASL/PLAIN unless you are on an already-encrypted network and you have a reason.
- **Authorization.** Kafka ACLs or Confluent RBAC. ACLs are simple: (principal, resource, operation, allow/deny) . For any cluster with more than one team, get ACLs on from day one — retrofitting them is painful.

A minimal durable config sketch:

```
listeners=SASL_SSL://:9094
security.inter.broker.protocol=SASL_SSL
sasl.enabled.mechanisms=SCRAM-SHA-512
sasl.mechanism.inter.broker.protocol=SCRAM-SHA-512

ssl.keystore.location=/etc/kafka/secrets/kafka.keystore.jks
ssl.keystore.password=***
ssl.truststore.location=/etc/kafka/secrets/kafka.truststore.jks
ssl.truststore.password=***

authorizer.class.name=org.apache.kafka.metadata.authorizer.StandardAuthorizer
super.users=User:admin
```

Monitoring

Kafka publishes everything via JMX. The usual Prometheus pattern is a JMX-exporter sidecar that scrapes MBeans and republishes them as Prometheus metrics. Strimzi ships this by default.

A minimal dashboard has, per broker:

- CPU, memory, disk space, network in/out, file-descriptor count.
- Under-replicated partitions, offline partitions, active controllers count (should be 1 cluster-wide).

- Request queue time p99, request handler idle percent.
- Bytes in / bytes out per second.

And per topic (aggregated cluster-wide):

- Messages-in-per-sec, bytes-in-per-sec, bytes-out-per-sec.
- Consumer group lag by group $\times$ topic.

Consumer lag is the single most valuable metric. It is the number of records between a consumer group's committed offset and the log-end offset. Lag growing unbounded is the first sign that a consumer can't keep up.

```
from confluent_kafka.admin import AdminClient

admin = AdminClient({"bootstrap.servers": "localhost:9092"})
groups = admin.list_consumer_groups().valid

for g in groups:
    offsets = admin.list_consumer_group_offsets([g.group_id])
    # ... compute lag per (topic, partition)
```

Quotas

Multi-tenant clusters benefit from *quotas*: limits on bytes-in / bytes-out / request rate per principal. A runaway producer can't starve the rest of the cluster if it's throttled to a fair share.

```
kafka-configs.sh --bootstrap-server localhost:9092 \
  --alter --add-config 'producer_byte_rate=10485760,consumer_byte_rate=20971520' \
  --entity-type users --entity-name alice
```

Alice is now capped at 10 MB/s in and 20 MB/s out. The broker returns throttle responses that the client library handles transparently; Alice sees latency climb, not failures.

Disaster recovery

The two common strategies:

- **MirrorMaker 2.** Replicate topics from one cluster to another. MM2 runs as Kafka Connect connectors and handles offset translation, topic provisioning, and ACL sync. Used for DR, for migrations, for hybrid cloud.
- **Stretched clusters.** A single cluster whose brokers live in multiple AZs or regions. Simpler to reason about (just one log) but cross-region latency kills write performance for most workloads.

For most teams, a second cluster replicated via MM2 is the right answer. Run regular restore drills. The first time you try MM2 during a real outage is the worst time to discover that a config was wrong.

Day-to-day runbooks

A handful of commands you'll use every week:

```
# Describe a topic (partitions, replicas, ISR)
kafka-topics.sh --bootstrap-server $BS --describe --topic $T

# List consumer groups and lag
kafka-consumer-groups.sh --bootstrap-server $BS --describe --group $G

# Reset a group to earliest/latest/a timestamp
kafka-consumer-groups.sh --bootstrap-server $BS --group $G \
  --reset-offsets --to-earliest --topic $T --execute

# Increase partitions (can't decrease!)
kafka-topics.sh --bootstrap-server $BS --alter --topic $T --partitions 12

# Check cluster metadata
kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
```

Keep a bookmark. These are the tools you reach for when something goes wrong.

Exercises

1. You have a three-broker cluster. One broker shows `UnderReplicatedPartitions = 847` and climbing. What do you check first? Second?
2. A producer gets throttled (slower acks, latency spikes) but no errors. What broker-side feature is almost certainly active? How do you see it in metrics?

3. You need to migrate a workload from one cluster to another with zero downtime. Sketch the MirrorMaker 2 + cutover plan in six steps.
4. A teammate wants to bump partition count from 12 to 36 on a live-production topic. What warning do you give them?

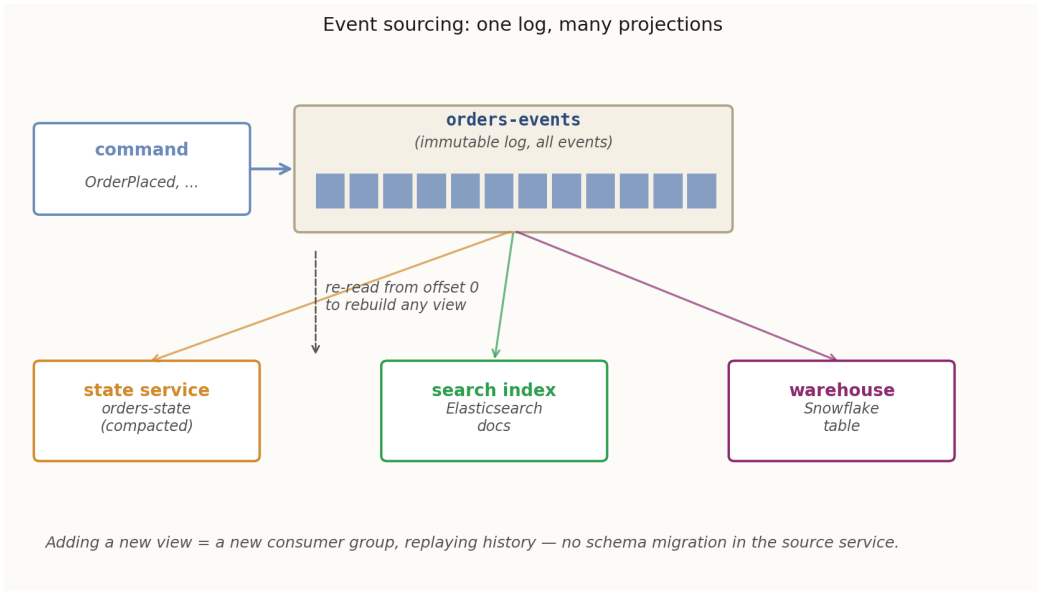
CHAPTER

Patterns That Keep Working

Ten chapters in, you have the pieces. This chapter is about the shapes that keep showing up when those pieces are wired into real systems. None of them are invented here; all of them are worth recognising, because reaching for the wrong one burns weeks.

Event sourcing

Instead of storing "what things look like now," you store "every change that has ever happened," and compute state by folding the log. Kafka is a natural substrate: the log is the source of truth; any view is a projection.



An event-sourced `orders` service: every business action is an event on `orders-events`; current state (`orders-state`) is a compacted view maintained by a consumer; new views can be built at any time by re-reading the event log.

Events on Kafka, keyed by aggregate ID, with a stable schema per event type:

```
events = [
    {"type": "OrderPlaced",    "order_id": "42", "items": [...]},
    {"type": "OrderPaid",     "order_id": "42", "amount": 9.99},
    {"type": "OrderShipped",  "order_id": "42", "carrier": "ups"},
]
```

A projection service reads the event topic and maintains a view — a PostgreSQL table, an Elasticsearch index, a Redis hash. New views are built by re-reading the log from offset 0. This is where Kafka's replayability earns its keep.

When to reach for this: domains with rich history (finance, healthcare, audit) and domains where "what did we think at 3pm yesterday?" is a real question.

When not to: CRUD apps where no one cares about history. You'll spend the same time and get less value.

CQRS

Closely related: Command-Query Responsibility Segregation. Writes (*commands*) go to one side — often a transactional database; events flow out via CDC or an outbox; reads (*queries*) hit a separate read-optimised store that is updated from the event stream.

The Kafka piece is the fan-out: one source of truth, many read models, each one maintained independently, each one free to be rebuilt.

Outbox

Services that need to update a database *and* publish an event have a consistency problem: what if one succeeds and the other fails? The outbox pattern solves it:

1. In one local transaction, write the business change *and* insert a row into an `outbox` table.
2. A separate process (or Debezium) reads new `outbox` rows and publishes them to Kafka.
3. The outbox row can be deleted once its event is safely on the topic.

```
BEGIN;  
UPDATE orders SET status='shipped' WHERE id=42;  
INSERT INTO outbox (topic, key, value)  
  VALUES ('orders-events', '42',  
          '{"type": "OrderShipped", "order_id": "42"}');  
COMMIT;
```

The database is the single source of truth for the atomic moment; Kafka publication is eventually-consistent but reliable. Debezium reading the outbox table via CDC is the industrial-strength version; a Python cron polling the table is the starter version.

Tip

The outbox pattern is the correct answer to "how do I publish an event and update a row atomically?" almost every time. Try to remember this before you invent a distributed transaction.

Compacted topic as a key-value store

A topic with `cleanup.policy=compact` keeps only the latest record per key. This is Kafka's answer to "I want the log to eventually collapse to the current state." Uses:

- **Changelog topics** behind stream processors (chapter 8).
- **Configuration distribution.** Service writes config updates; all consumers rebuild their config by reading the topic.
- **Shared lookup tables.** CDC-fed product catalog topic, consumed as state by every service that needs it.

```
kafka-topics.sh --bootstrap-server $BS --create \  
  --topic product-catalog --partitions 12 --replication-factor 3 \  
  --config cleanup.policy=compact \  
  --config min.cleanable.dirty.ratio=0.1 \  
  --config segment.ms=3600000
```

A deletion in the logical table becomes a *tombstone* — a record with a null value. Compaction drops every earlier record for that key, then eventually drops the tombstone itself.

Warning

Compacted topics do not give you "every record for this key in order." They give you "the latest record for each key." If you need history, use a delete-retention topic instead, or both.

Dead-letter queues

Some records can't be processed. A schema the consumer doesn't understand, a downstream system that's refusing a specific payload, a bug in `process()`. The at-least-once-plus-retry model doesn't help: the record keeps coming back forever. A *dead-letter topic* is where such records go to live out their days while the consumer continues.

```
try:
    process(msg)
except UnrecoverableError as e:
    dlq_producer.produce("orders-dlq",
                        key=msg.key(),
                        value=msg.value(),
                        headers=[("error", str(e).encode()),
                                ("source", msg.topic().encode()),
                                ("offset", str(msg.offset()).encode())])
    c.commit(message=msg, asynchronous=False)
```

Three rules for DLQs that age well:

1. **Keep the headers.** Record the reason for rejection, source topic, original offset. You'll thank yourself at 3 AM.
2. **Alarm on DLQ rate, not absolute count.** A growing DLQ is an incident; a flat low-volume one is a data-quality warning.
3. **Have a replay plan.** A DLQ with no replay tooling is a data trash can. Invest in the tooling *before* you need it.

Saga pattern

Long-running business processes spanning multiple services are better modelled as a sequence of local events, each compensating for the previous one on failure, rather than as a distributed transaction:

- `OrderPlaced` → reserve inventory
- `InventoryReserved` → charge card
- `CardCharged` → ship order
- Any step fails → emit a compensating event, unwind the earlier steps.

Each service is a simple consumer-plus-producer. No two services share a transaction. The log is the coordination layer.

Exactly-once stream processing

Revisited from chapter 6: a consume-transform-produce loop, all transactional. This is the pattern under the hood of ksqldb, Kafka Streams, and a well-configured Faust. If you build

it yourself, copy the skeleton from chapter 6 and resist the temptation to deviate.

Tiered storage

Recent feature, but increasingly standard: offload old log segments to cheap object storage (S3, GCS) while keeping recent ones on broker disk. Your retention becomes effectively unlimited; broker disks shrink; reads from cold data are slower but still work. Enable per topic:

```
kafka-topics.sh --alter --topic events \  
  --config remote.storage.enable=true \  
  --config local.retention.ms=86400000 \  
  --config retention.ms=31536000000
```

1 day hot
1 year total

For any topic where you want a year of history but the cluster only has a month of fast disk, this is the pattern.

The anti-patterns

Briefly, shapes that look appealing but hurt:

- **"Kafka as a request-response queue."** If you need a reply, Kafka makes you invent correlation IDs, consumer groups for replies, and timeouts — all of which HTTP gives you for free.
- **"One gigantic topic for everything."** Consumers then subscribe to the firehose and filter. You pay the full bandwidth for every consumer, and you can't easily scale one workload without the others.
- **"Partition by timestamp."** Time-keyed partitioning creates a hot partition at the present moment; older partitions go idle. Almost always the wrong choice; key by entity instead.
- **"Store blobs in Kafka values."** Kafka's tight loop isn't tuned for megabyte records. Write blobs to S3; write the S3 key to Kafka.

Exercises

1. A legacy service updates a Postgres row and fires an HTTP webhook. The webhook sometimes fails, leaving the external system out of sync. Describe the outbox-pattern fix, including which table changes are needed.
2. A product team builds a shopping-cart feature with four events (`ItemAdded` , `ItemRemoved` , `CartCheckedOut` , `CartAbandoned`). Give a partitioning key that preserves ordering and argue why.
3. Your team's orders topic has grown to 10 TB. Operations wants to shrink broker disks. Which pattern from this chapter helps, and what is the migration plan?
4. Sketch a DLQ replay tool in 30 lines of Python: it reads the DLQ topic and re-publishes each record back to the original topic, one at a time, with a `retry-count` header.

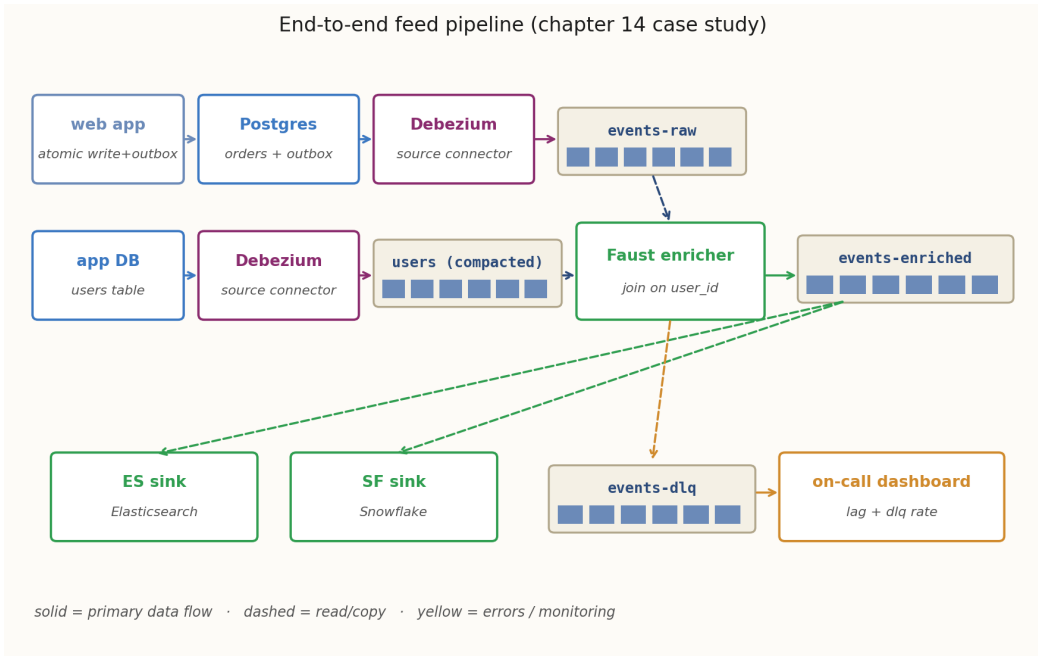
CHAPTER

A Case Study: The Feed Pipeline

Everything up to this point has been pieces. This chapter wires them together end-to-end. We build a small but complete system: an activity feed that ingests user events from a web app, enriches them against a user-profile service, and pushes the result to a search index and a warehouse.

It is small enough to fit in one chapter, and it exercises every idea the book has touched: keys and partitions, idempotent producers, consumer groups, CDC via Connect, a stream-processing enrichment, an outbox pattern, a dead-letter topic, and a monitoring story.

The shape



The feed pipeline: web app writes events via an outbox; Debezium streams the outbox to `events-raw`; Faust enriches against a compacted `users` topic and writes `events-enriched`; sinks push that to Elasticsearch and Snowflake; a DLQ catches the stragglers.

Seven moving parts:

1. **Web app** — writes events to an outbox table in its Postgres.
2. **Debezium** — CDC from Postgres outbox to `events-raw` topic.
3. `events-raw` — immutable firehose of user actions.
4. `users` — compacted topic of current user profiles.
5. **Faust enricher** — joins `events-raw` × `users` → `events-enriched`.
6. **ES sink** — Connect pushing `events-enriched` into Elasticsearch.
7. **Snowflake sink** — Connect pushing `events-enriched` into Snowflake.

Plus operational topics: `events-dlq` for records the enricher can't handle, and `_schemas` for the schema registry.

Step 1: event envelope

Every topic uses an Avro envelope with a stable header and a variable payload. A single schema on the outside means one deserialiser in every consumer; the payload is where business evolution happens.

```
{
  "type": "record",
  "name": "Event",
  "fields": [
    {"name": "id", "type": "string"},
    {"name": "occurred_at", "type": "long", "logicalType": "timestamp-millis"},
    {"name": "type", "type": "string"},
    {"name": "user_id", "type": "string"},
    {"name": "payload", "type": "bytes"}
  ]
}
```

The `payload` field holds an inner Avro-serialised record whose schema ID is carried in the outer record's headers. Consumers dispatch on `type` and deserialise `payload` against the registered schema. This is the pattern the schema registry's *Event Router* support encodes; rolling your own is ~50 lines.

Step 2: outbox + CDC

The web app's Postgres has an `outbox` table:

```
CREATE TABLE outbox (
  id          BIGSERIAL PRIMARY KEY,
  key         TEXT       NOT NULL,
  topic       TEXT       NOT NULL DEFAULT 'events-raw',
  value       BYTEA      NOT NULL, -- Avro-serialised Event
  created_at  TIMESTAMPTZ DEFAULT now()
);
```

Every business transaction inserts an `outbox` row alongside its domain write:

```
def place_order(session, order):
    session.execute(text("""
        INSERT INTO orders (id, user_id, total) VALUES (:id, :uid, :t)
    """), dict(id=order.id, uid=order.user_id, t=order.total))
    session.execute(text("""
        INSERT INTO outbox (key, value)
        VALUES (:key, :val)
    """), dict(key=str(order.id),
               val=serialise_event("OrderPlaced", order)))
```

Debezium watches the `outbox` table and publishes each new row to `events-raw`, keyed by `key`. The web app never talks to Kafka directly. The database is the sole atomic boundary.

Step 3: user state, compacted

The `users` topic is populated by a separate Debezium connector watching the `users` table. `cleanup.policy=compact` so it is a current-state lookup: the latest record per `user_id` is always there.

```
kafka-topics.sh --create --topic users --partitions 12 \
  --config cleanup.replication.factor=3 \
  --config cleanup.policy=compact
```

Profile updates land as records; deletes become tombstones.

Step 4: the enricher

A Faust app that reads `events-raw`, looks up the `users` table by `user_id`, and writes an enriched record to `events-enriched`:

```

import faust

app = faust.App("enricher", broker="kafka://kafka:9092")

events = app.topic("events-raw", key_type=str, value_type=bytes)
users = app.topic("users", key_type=str, value_type=bytes)
enriched = app.topic("events-enriched")
dlq = app.topic("events-dlq")

user_table = app.Table("user-state", default=lambda: None,
                       partitions=12)

@app.agent(users)
async def materialise_users(stream):
    async for user_id, value in stream.items():
        user_table[user_id] = value  # None value = tombstone

@app.agent(events)
async def enrich(stream):
    async for ev in stream:
        try:
            user = user_table[ev.user_id]
            if user is None:
                await dlq.send(key=ev.id, value=encode(ev, "no_user"))
                continue
            await enriched.send(key=ev.id, value=enrich(ev, user))
        except Exception as e:
            await dlq.send(key=ev.id, value=encode(ev, str(e)))

if __name__ == "__main__":
    app.main()

```

The important properties:

- **Keyed the same way.** `events-raw` and `users` are both keyed by `user_id` (or in `events-raw`, the event's `id` — the enricher re-keys by user before the join). Same key, same partition, same processing member: the join is local.
- **Faust owns the state.** `user_table` lives in RocksDB on the enricher's local disk, with a changelog backup to a compacted internal topic.
- **DLQ is explicit.** Anything that can't be enriched goes to `events-dlq` with a reason, and processing continues.

Step 5 & 6: the sinks

Connect runs two sink connectors:

```
requests.post("http://connect:8083/connectors", json={
    "name": "es-feed",
    "config": {
        "connector.class":
            "io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
        "topics": "events-enriched",
        "connection.url": "http://es:9200",
        "key.ignore": "false",
        "schema.ignore": "true",
        "tasks.max": "6",
    },
})

requests.post("http://connect:8083/connectors", json={
    "name": "sf-feed",
    "config": {
        "connector.class":
            "com.snowflake.kafka.connector.SnowflakeSinkConnector",
        "topics": "events-enriched",
        "snowflake.url.name": "...",
        "buffer.count.records": "10000",
        "buffer.flush.time": "60",
        "tasks.max": "4",
    },
})
```

The Elasticsearch sink maintains a per-user document keyed by `event id` ; queries in the UI become one-hop lookups. The Snowflake sink accumulates batches every 60 seconds into a staging table the warehouse consolidates nightly.

Step 7: monitoring

Per-topic lag dashboards for each consumer group (Faust, Connect), plus:

- `events-dlq` send rate — should be < 0.1% of `events-raw` send rate.
- `events-enriched` bytes-out — should track bytes-in on `events-raw` .
- Enricher process CPU, RocksDB size, rebalances per hour.

A single Grafana dashboard with these panels is enough to know whether the pipeline is healthy without looking at anything else.

Failure walk-through

How does this system behave under each failure?

- **Web app crashes mid-transaction.** The domain and outbox writes are one transaction: either both or neither. Debezium doesn't see a half-write.
- **Broker reboots.** Producers (including Debezium) reconnect; consumers reconnect; in-flight work re-reads from committed offsets; duplicates don't reach the sinks because idempotent producers + idempotent sinks.
- **Enricher pod OOMs.** Faust restarts, restores `user_table` from changelog, resumes from last committed offset in `events-raw`. A few seconds of replay, no loss.
- **A malformed event.** Enricher catches the exception, routes to `events-dlq`, commits the `events-raw` offset, continues. An on-call dashboard on DLQ rate triages later.
- **Elasticsearch outage.** Sink connector pauses; lag grows; when ES comes back, sink catches up. No data loss as long as `events-enriched` retention is longer than the outage.

Each answer comes back to the same two properties: per-partition ordering, and at-least-once + idempotency.

Numbers

A production instance of this pipeline, at steady state, typically looks like:

- `events-raw` — 20k records/s, 60 MB/s, 12 partitions.
- `users` — 100 records/s (profile updates), compacted.
- `events-enriched` — 20k records/s, 70 MB/s (wider records).
- Enricher — 6 pods, each at ~25% CPU; RocksDB at ~8 GB.
- ES sink — 6 tasks, ~500 ms p99 index time.
- Snowflake sink — 4 tasks, one COPY every 60 s.
- End-to-end median latency (web app write to ES-readable): ~8 s.

The ~8 seconds is a design decision: 60-second Snowflake batches don't help, but Elasticsearch could be lower if we shrank Connect's internal flush interval. "Real-time enough" is usually what matters.

What this skips

- Security. Production adds TLS, mTLS or SASL/SCRAM, ACLs per service, and a secrets story.
- Multi-region. DR via MirrorMaker 2 to a second cluster in another region. Active-active is possible but non-trivial.
- Backfills. The first time you onboard a new consumer of `events-enriched`, you replay from offset 0. Design for it.

But the shape is complete. Every paragraph of the book before this chapter shows up somewhere in the diagram.

Exercises

1. The enricher's `user_table` has one entry per user, ~1 KB each, for 50M users. Estimate the RocksDB footprint. Does this fit on a single pod?
2. Redesign the pipeline without the outbox: the web app produces directly to Kafka. In one paragraph, name two specific things that get worse.
3. A downstream team wants a stream of enriched events for *new users only* (within the first 7 days of signup). Sketch where the logic belongs — in the enricher, in a new stream-processing service, or in the consumer — and argue why.
4. A bug has corrupted 2 hours of events on `events-enriched`. Work out how to rebuild that window from `events-raw` and the `users` topic, without disturbing live traffic.

CHAPTER

Appendix A: The CLI Cheatsheet

The Kafka distribution ships a handful of shell scripts that wrap the Java admin client. They are the fastest way to probe or alter a cluster. This appendix is a one-stop reference for the commands you'll reach for most.

All examples assume a bootstrap server on `localhost:9092` ; set an env var once and forget it:

```
export BS=localhost:9092
```

Topics

```
# List all topics
kafka-topics.sh --bootstrap-server $BS --list

# Describe one topic (partitions, replicas, ISR, leaders)
kafka-topics.sh --bootstrap-server $BS --describe --topic orders

# Create a topic
kafka-topics.sh --bootstrap-server $BS --create --topic orders \
  --partitions 12 --replication-factor 3

# Create a compacted topic
kafka-topics.sh --bootstrap-server $BS --create --topic users \
  --partitions 6 --replication-factor 3 \
  --config cleanup.policy=compact

# Increase partitions (CANNOT shrink!)
kafka-topics.sh --bootstrap-server $BS --alter --topic orders \
  --partitions 24

# Change retention on an existing topic
kafka-configs.sh --bootstrap-server $BS --alter --entity-type topics \
  --entity-name orders --add-config retention.ms=604800000

# Delete a topic
kafka-topics.sh --bootstrap-server $BS --delete --topic scratch
```

Consumer groups

```
# List all consumer groups
kafka-consumer-groups.sh --bootstrap-server $BS --list

# Describe a group's current offsets and lag
kafka-consumer-groups.sh --bootstrap-server $BS --describe \
  --group orders-ui

# Reset offsets to earliest / latest / by timestamp / by offset
kafka-consumer-groups.sh --bootstrap-server $BS --group orders-ui \
  --reset-offsets --topic orders --to-earliest --execute

kafka-consumer-groups.sh --bootstrap-server $BS --group orders-ui \
  --reset-offsets --topic orders --to-datetime 2026-01-01T00:00:00.000 \
  --execute

kafka-consumer-groups.sh --bootstrap-server $BS --group orders-ui \
  --reset-offsets --topic orders --to-offset 12345 --execute

# Delete a consumer group
kafka-consumer-groups.sh --bootstrap-server $BS --delete \
  --group orders-ui-old
```

Warning

Resetting offsets on a live consumer group with members currently running is undefined behaviour. Stop the consumers, reset, then restart.

Producing and consuming from the shell

Useful for quick tests without writing a Python script.

```
# Produce – type records on stdin, one per line
kafka-console-producer.sh --bootstrap-server $BS --topic orders \
  --property "parse.key=true" --property "key.separator=:"

# e.g. type: 42:{"item":"coffee"}<enter>

# Consume – tail from the end
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --property "print.key=true" --property "key.separator= → "

# Consume from the beginning
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --from-beginning --max-messages 10
```

Cluster metadata

```
# Which brokers are alive?
kafka-broker-api-versions.sh --bootstrap-server $BS

# Describe the KRaft quorum (controllers)
kafka-metadata-quorum.sh --bootstrap-server $BS describe --status

# See the controller's voter list
kafka-metadata-quorum.sh --bootstrap-server $BS describe --replication
```

Partition reassignment

```
# 1. List topics to move
cat > topics.json <<EOF
{"topics": [{"topic": "orders"}], "version": 1}
EOF

# 2. Generate a proposed assignment across brokers 1,2,3,4
kafka-reassign-partitions.sh --bootstrap-server $BS \
  --topics-to-move-json-file topics.json \
  --broker-list "1,2,3,4" --generate > proposal.json

# 3. Apply it, throttled at 50 MB/s
kafka-reassign-partitions.sh --bootstrap-server $BS \
  --reassignment-json-file proposal.json \
  --throttle 52428800 --execute

# 4. Check progress
kafka-reassign-partitions.sh --bootstrap-server $BS \
  --reassignment-json-file proposal.json --verify
```

ACLs (authorisation)

```
# Allow principal "alice" to produce to topic "orders"
kafka-acls.sh --bootstrap-server $BS --add \
  --allow-principal User:alice \
  --operation Write --topic orders

# And to read it, in the "analytics" consumer group
kafka-acls.sh --bootstrap-server $BS --add \
  --allow-principal User:alice \
  --operation Read --topic orders \
  --operation Read --group analytics

# List ACLs for a principal
kafka-acls.sh --bootstrap-server $BS --list --principal User:alice
```

Configurations

Kafka has dozens of tunables at broker, topic, user, and client levels. Dump them with:

```
kafka-configs.sh --bootstrap-server $BS --describe \  
  --entity-type brokers --entity-name 1  
  
kafka-configs.sh --bootstrap-server $BS --describe \  
  --entity-type topics --entity-name orders --all
```

The `--all` flag shows both explicit and default values — useful for understanding what a topic is actually running with.

GetOffsetShell

A quick way to get the log-end offset of each partition:

```
kafka-get-offsets.sh --bootstrap-server $BS --topic orders --time -1
```

`--time -1` = latest, `--time -2` = earliest, or pass a millisecond timestamp to ask "what was the earliest offset after time T?"

From Python

When you don't want to shell out, the same operations are available through `confluent_kafka.admin.AdminClient`:


```

from confluent_kafka.admin import (AdminClient, NewTopic, ConfigResource,
                                   ResourceType)

admin = AdminClient({"bootstrap.servers": "localhost:9092"})

# List topics
md = admin.list_topics(timeout=5)
for name in md.topics:
    print(name)

# Create a compacted topic
fut = admin.create_topics([
    NewTopic("users", num_partitions=6, replication_factor=3,
            config={"cleanup.policy": "compact"})
])
for name, f in fut.items():
    f.result()    # raises on error

# Read a topic's current config
r = ConfigResource(ResourceType.TOPIC, "orders")
cfg = admin.describe_configs([r])[r].result()
for k, v in cfg.items():
    print(k, "=", v.value)

```

Useful when a management UI or one-off script needs to stay in Python rather than shell out to the Kafka binaries.

CHAPTER

Appendix B: Glossary

Terms you'll meet in the Kafka docs and in this book, with one-paragraph definitions aimed at making the jargon less opaque.

acks. A producer config that controls what "the record is written" means. `0` = don't wait, `1` = wait for the leader's disk, `all` = wait for every in-sync replica. `all` with `min.insync.replicas>=2` is the durable choice.

broker. A single Kafka server. Brokers hold partitions and replicate them to each other. A production cluster has at least three.

changelog topic. A compacted internal topic that stream-processing libraries use to back their state stores. When a worker restarts, it replays the changelog to rebuild state.

compacted topic. A topic with `cleanup.policy=compact`. Kafka keeps only the latest record per key, deleting older records for the same key. Behaves like an eventually-consistent key-value view.

consumer group. A named set of consumer processes that share the work of reading a topic. Each partition is assigned to exactly one member at a time. Scaling the group scales parallelism up to the partition count.

controller. A broker that owns cluster metadata: which broker is leader for which partition, which topics exist, which ACLs are set. Under KRaft, the controller role is a Raft group inside the brokers.

CDC (change data capture). Streaming a database's write-ahead log as Kafka records. The dominant tool is Debezium. CDC is how a relational database becomes an event source.

delivery report. The callback that fires when a produced record is acknowledged (or fails) by the broker. Accessed via `on_delivery` in `confluent-kafka-python`, or `future.get()` in `kafka-python`.

fetch. A consumer's request to a broker for new records on a partition. Bounded by `fetch.min.bytes`, `fetch.max.bytes`, `fetch.max.wait.ms`.

HWM (high-water mark). One past the highest offset that every in-sync replica has written. Consumers can only read up to the HWM. Protects readers from seeing records that might later be lost in a leader failover.

idempotent producer. A producer with `enable.idempotence=true` that attaches a producer ID and per-partition sequence number to each batch, so the broker can drop duplicates caused by retries.

in-sync replica (ISR). A replica that is caught up with the leader within `replica.lag.time.max.ms`. The ISR set shrinks and grows as brokers lag or catch up.

KRaft. "Kafka Raft," Kafka's built-in consensus protocol that replaces ZooKeeper. Default in 3.5+; production-ready since 3.3. Makes the cluster a single-service affair.

leader. The broker that handles all reads and writes for a given partition. Followers replicate from the leader. Leadership can move between replicas; membership in the replica set is static until reassignment.

log-end offset (LEO). One past the highest offset on a partition's leader. The offset the next produce will get.

log-start offset. The lowest offset still present on disk. Older records have been deleted by retention policy.

min.insync.replicas. A topic-or-broker config that caps how durable an `acks=all` produce is. If the ISR shrinks below this, `acks=all` produces fail with `NotEnoughReplicas`.

offset. A 64-bit integer identifying a record within a partition. Offsets are dense and monotonic per partition; they mean nothing across partitions.

outbox pattern. A design where a service atomically writes its domain change and an event-publication row to its database, and a separate process (usually Debezium) relays the event to Kafka. Solves the "update DB and publish event atomically" problem.

partition. An ordered, append-only sequence of records living on one broker (with replicas on others). Kafka's unit of parallelism, ordering, and placement.

rebalance. The process of reassigning partitions among the members of a consumer group. Triggered by join, leave, or session timeout. *Cooperative rebalancing* only moves partitions whose owner has changed; *eager rebalancing* pauses everyone.

replication factor (RF). The number of copies of a partition. RF=3 is standard; RF=1 is development-only.

retention. How long (or how much) to keep records. Topic-level configs `retention.ms` and `retention.bytes`. Old segments are deleted; they are not rewritten.

schema registry. A service that stores schemas (Avro, Protobuf, JSON Schema) centrally and enforces compatibility rules on updates. Producers look up or register schemas; consumers look them up to deserialise. The Confluent Schema Registry is the dominant implementation.

segment. A file on disk that holds a contiguous range of offsets for a single partition. A partition is a sequence of segments; once a segment reaches `log.segment.bytes`, it closes and a new one opens.

sticky partitioner. A producer-side optimisation that routes keyless records to the same partition for a short window, improving batching. You don't configure it; modern clients use it by default.

tombstone. A record with a null value on a compacted topic. Signals "delete this key." Eventually the compactor drops both the earlier records for the key and the tombstone itself.

topic. A named stream of records, split across one or more partitions.

transactional producer. A producer with a `transactional.id` set, capable of grouping several produces and one offset commit into a single atomic unit. The piece that makes exactly-once stream processing possible.

under-replicated partition (URP). A partition whose ISR is smaller than its full replica set. One URP is usually recoverable; lots of URPs is an incident.

unclean leader election. Allowing an out-of-sync replica to be elected leader when all in-sync replicas are gone. Trades durability for availability. Off by default, and the default is right.

ZooKeeper. The legacy metadata service Kafka used before KRaft. Still supported; no longer needed. New deployments use KRaft.